Ruminant Farm System Model

Soil and Crop

June 23, 2025

v.1; Updated 2025

Contents

List of Figures

List of Tables

I. Preface: A Day in the Life on a RuFaS Farm

This document provides a high-level overview of the daily operations of a ruminant farm, as modeled by RuFaS. The overview will be through the eyes of Dr. Rue Fas. One day, Dr. Rue woke up and decided the life of a computer scientist wasn't for her. So she bought a dairy farm, and this is how she operates it every day.

Feeding the Animals Second thing in the morning, after coffee, Dr. Rue checks the calendar to see if it's time to formulate a new ration for her animals.

If a new ration needs to be formulated, Dr. Rue heads over to the barn where she keeps all her forages and feed. While there, she checks how well all the forages are holding up. After considering the feeds available, the quality of each one, and how much each one costs, she decides on how productive she wants her animals to be. It is a difficult balancing act, but eventually a ration recipe is found that will keep the animals happy while keeping costs low and sustainability high until the calendar says the ration recipe needs to be reformulated.

When Dr. Rue doesn't need to formulate a new ration, she takes the last ration recipe she made and grabs all the feed and forage the animals need for the day out of the feed barn. If there isn't enough for the animals, then a new ration is formulated using the steps above.

Taking Care of the Animals After making sure all the animals' feed is taken care of, Dr. Rue moves on to taking care of all the other animal tasks. This includes milking the animals, making sure they're healthy, getting them pregnant, and numerous other tasks.

As dawn breaks on the RuFaS farm, the animals awaken after a restful night. Joe, the farm master, prepares a large whiteboard to track HerdReproductionStatistics, ensuring he has all the data he needs for the day's activities.
Daily Routine - After leaving their pens, each animal lines up in front of the barn. Julie, the diligent farm veterinarian, examines them one by one. This daily check includes updates on the following areas:

- Nutrients: Each animal begins its day with a quick phosphorus requirement assessment. Julie records these details and updates the animal's phosphorus-related metrics.
- Digestive System: With nutrient information in hand, Julie shifts focus to each animal's digestive system, estimating manure output and enteric methane production for that day.
- Milk Production: All milking cows are assessed to gauge daily milk yield and milk component content (fat and protein). Any necessary adjustments are noted.
- Animal Growth: Around midday, Julie returns to each animal to evaluate daily weight changes. She updates each animal's body weight accordingly.
- Reproduction: HeiferII and Cow stage animals undergo a reproduction check. Julie closely monitors their reproductive progress and updates the HerdReproductionStatistics details on Joe's whiteboard.
- Life Stage Evaluation: Once the daily routines are complete, Julie determines if any animal is ready to progress to the next life stage. Graduating animals are marked as "graduated" to be moved to their new pens. Animals that must leave the herd—due to health, productivity, or other issues—are labeled as "sold" or "dead" to be removed from the farm population.
- Herd Size Maintenance: After every animal has been evaluated, Joe counts the herd to ensure it remains at a healthy size. If it is too large, he arranges the sale of some animals. If the herd is too small, he may opt to purchase additional heiferIIIs to keep numbers at the ideal level.
- Herd Structure Management: Joe also manages pen assignments. Newly graduated or newborn animals are guided to suitable pens, while animals marked for removal (sold or deceased) are taken out of the herd and removed from the farm records.

Output Reporting At day's end, Joe and Julie review the updated HerdReproductionStatistics on the whiteboard. Joe then compiles a concise report summarizing the day's herd stats. He forwards this information to the OutputManager before wrapping up another successful day on the RuFaS farm.

Dealing with the Manure Once all the animal chores are finished, Dr. Rue moves on to handling the manure. There are a lot of different ways that manure is collected from the animals: it is flushed out of the milking parlor with water, some of the pens need to have it shoveled out along with the bedding in the pen, and some pens have grates that allow the manure to go straight into a pit beneath the barn.

There are a lot of different places where manure is processed and stored on the farm. Dr. Rue goes from place to place on the farm, moving the manure from one place to the next. The manure chores are finished after moving all of the manure into her lagoons, compost piles, and other storages. As this final manure chore is completed, she writes down all the information about how much manure is in all these storages, and some information about what that manure is made of. How much water, nitrogen, phosphorus, other solids, etc. are in each storage.

Managing the Fields Now that Dr. Rue is done making sure all the manure has been handled and stored properly, she can take care of all the field tasks. She has a lot of fields on her farm, each with its own unique schedule. Her field management routine starts with checking each field's schedule to see if any of them are due to have manure applied, and if so, how much they should have applied. After noting all the manure that is supposed to go onto her fields, these amounts are checked against the amounts of manure that are in her manure storage. Dr. Rue then applies manure to each field on the schedule that day, trying to match the amount applied to the amount on the schedule as best she can.

After making sure every manure item on each farm's schedule has been taken care of for the day, the schedule for each field is rechecked, one field at a time. In each schedule she checks to see if that field needs to be tilled, if any chemical fertilizer needs to be applied, and if any crops need to be planted or harvested. After all these activities are taken care of, Mother Nature goes to work. The sun shines and rain falls, providing the field with water and energy that grows crops and soaks the soil. After taking care of each field one by one, Dr. Rue drives back to the main farm with all of the crops that she harvested on that day.

Storing the Feeds Now that she is back at the barns, all the crops that were just harvested can be stored. Fortunately she is very organized, so all her harvested crops are separate from one another and each has a label, indicating where it will be stored inside the feed barn. As each feed is stored, it is labeled with the current date. Dr. Rue is exhausted from another long day. She goes back to the farmhouse and rests up, preparing to do the whole thing over again tomorrow. Her daily routine might seem repetitive, but each day is a crucial part of the larger, continuous process that keeps her farm running efficiently and sustainably.

II. Module Introduction

The Soil and Crop Module simulates the daily changes in soil composition and crop growth based on nutrient availability, weather and soil types. The underlying biophysical processes are drawn from existing models of soil and plant biogeochemistry including the SWAT model, Daycent, and SurPhos. The management routines execute the daily biogeochemical processes and direct simulation of the user-provided management practices including application of manure or synthetic fertilizer, tillage, planting and harvesting. As part of the whole farm biophysical simulation in RuFaS, the Soil and Crop Module receives manure from the Manure Module and sends harvested crop biomass to the Feed Storage Module.

III. Field Management

A. Introduction

The Field Manageme (FieldManager) is analogous to the agronomist or field manager on a real dairy farm. It coordinates and manages all the different routines and processes that are executed in Soil and Crop Module across any number of fields. To start with, the field manager interprets and integrates all of the user's inputs about field management, crops, and soils to create a set of fields that each have their specified field and soil properties as well as schedules for each of the key management operations (manure application, fertilizer application, tillage, planting, and harvesting).

Once the fields and their schedules are set up, the FieldManager works through daily updates for each field. These daily updates include:

- Gathering the weather conditions for that day
- Checking if there is a manure application needed so that it can coordinate with Manure Module to provide the manure for the application
- Calling each field's daily update methods
- Recording the harvested crops and sending them to the Feed Storage Module

The FieldManager is also responsible for calling functions to record and reset any annual data records or processes for each field. Thus, the FieldManager is responsible for receiving and generating information from the user and other parts of the model and coordinating the simulation within the Soil and Crop Module.

Setting up Field Management RuFaS can simulate as many fields as the user wishes to specify and requires a unique input file for each field. However, individual fields can share soil, crop, and management schedule inputs so that the user can simulate multiple fields with, for example, the same soil configuration but different crop and management schedules or different crops with the same manure application schedules without having to replicate the inputs for fields with shared characteristics.

Field Management Initialization The FieldManager class, implemented in field_manager.py, has a series of functions to help package the user-provided configuration data and management schedule information required to initialize each field. These functions all start with the prefix "set_up" and are called by the wrapper function "set_up_field()." There is only one instance of the FieldManager class as it is used to set up and coordinate processes across all the fields. The following functions within the FieldManager class gather information and call other functions to help initialize key data classes required for initialization of each field:

- "setup_field_data": This function sets up the field data parameters and returns an instance of the FieldData class populated with the values from the field_configuration_data
- "setup_soil_data": This function sets up a Soil instance that itself contains a SoilData instance configured to the user-provided specifications
- "setup_soil_layer_data": This function sets up a soil LayerData instance configured with provided data that is added to the SoilData instance

See the Individual Field Management section below for more information about the FieldData Class and the Soil submodule documentation for information related to the soil data classes.

The next group of "set_up" functions in FieldManager help initialize the management schedules for each field. These functions call initialization routines for a schedule class that is used to generate an instance of a schedule with all the information that is specific to each type of management practice for each field. These schedule classes translate the user input into detailed schedules that can be read and interpreted by the FieldManager and instances of RuFaS Fields on a daily basis to determine the operations that need to be simulated each day. The schedule set up functions are:

- "set_up_fertilizer_events": This function generates fertilizer application event schedules based on the user-provided fertilizer schedule and returns a fertilizer schedule as well as a dictionary with the available fertilizer mixes
- "set_up_manure_events": This function generates manure application event schedules based on the user-provided manure application schedules and returns a list of the planned manure applications.
- "set_up_tillage_events": This function generates tillage event schedules based on the user-provided tillage schedules and returns a list of the planned tillage events.
- "set_up_crop_events": This function generates all planting and harvest events based on the user-provided crop rotation schedule and returns a list of planting events and a list of the harvesting events that correspond to the planting events.
- "set_up_crop_schedules": This function is called by set_up_crop_events() and returns a single list of all the cropping schedules.

Outputs for each field can be identified by the character string used to define the field input blob in the metadata file.
For example, in the below excerpt from a metadata input file:

```
"field_example_1": {
  "title": "Field specification",
  "description": "Field characteristics and references to field management specifications.",
  "path": "input/data/field/example_field_1.json",
  "type": "json",
  "properties": "field_properties"
},

"field_example_2": {
  "title": "Field specification",
  "description": "Field characteristics and references to field management specifications.",
  "path": "input/data/field/example_field_2.json",
  "type": "json",
  "properties": "field_properties"
},
```

Figure 1: Example of the metadata input file.

there are 2 different fields named "field_example_1" and "field_example_2" so the outputs generated by the first field would take the form:
Class.function.variable.field='field_example_1'
Where the pre-fix Class.function.variable is determined by the class, function that produced the variable, and the variable being reported. As a specific illustration, the output variable for the daily surface residue from "field_example_1" would be:
FieldDataReporter.send_field_daily_variables.current_residue.field='field_example_1'

Field Setup and Initialization When directed by the FieldManager, the Field class initializes a new Field object with the user-provided information that is packaged and shared by the FieldManager. These inputs include things like the field location, size, irrigation practices, pointers to the input files with management schedules, and options for turning crop growth stressors on and off.

Required Inputs

Table 1: Required user inputs for the FieldManager section.

Section	Variable	Definition	Default	Min	Max
Field	field_size	Size of the field (ha)	N/A	0	NA
	absolute_latitude	The absolute latitude of the center of this field.(degrees).	43.5	0	90
	longitude	The longitude of the center of this field (degrees).	-89.4	-180	180
	watering_amount_in_liters	Amount of water to be applied as irrigation at the end of a specified interval (L).	0	0	N/A
	watering_interval	Number of days that make up the irrigation interval between irrigation events (d).	0	0	N/A
	simulate_water_stress	Whether water stress affects crops grown in this field.	TRUE	N/A	N/A
	simulate_temp_stress	Whether temperature stress affects crops grown in this field.	TRUE	N/A	N/A
	simulate_nitrogen_stress	Whether nitrogen stress affects crops grown in this field.	TRUE	N/A	N/A
	simulate_phosphorus_stress	Whether phosphorus stress affects crops grown in this field.	TRUE	N/A	N/A
Crop Management	crop_species	Name of the crop being grown.	Must be selected from names of configurations defined in the crop configuration input file.		
	harvest_days	Julian day(s) of year to harvest	N/A	1	366

Continued on next page

Table continued from previous page

Section	Variable	Definition	Default	Min	Max
	harvest_years	Calendar years in which the harvesting occurs.	N/A	1	
	harvest_operations	Operation(s) with which this crop will be harvested.	-	• harvest_only • harvest_kill • kill_only	
	harvest_type	Whether the crop uses scheduled harvests or optimal harvests (“scheduled”, “optimal”).	scheduled		
	planting_days	Julian day(s) of year to plant.	N/A	1	366
	planting_years	Calendar years in which the planting occurs.	N/A	1	
	pattern_repeat	Number of times that this crop schedule should be repeated.	0	0	N/A
	planting_skip	Number of years to be skipped between planting schedule repetitions.	0	0	N/A
	harvesting_skip	Number of years to be skipped between harvest schedule repetitions.	0	0	N/A
Fertilizer Management	name	Name of the fertilizer mix.			
	N	Fraction of nitrogen contained in the fertilizer mix by mass.	1	0	1
	P	Fraction of phosphorus contained in the fertilizer mix by mass.	1	0	1

Continued on next page

Table continued from previous page

Section	Variable	Definition	Default	Min	Max
	K	Fraction of potassium contained in the fertilizer mix by mass.	1	0	1
	ammonium_fraction	Fraction fertilizer nitrogen that is ammonium. All non-ammonium nitrogen is assumed to be nitrate.	0.5	0	1
Fertilizer Schedule	mix_names	List of the mix names that will be used for the corresponding fertilizer application.			
	years	List of years in which fertilizer will be applied.		1	
	days	List of days on which fertilizer will be applied.		1	366
	nitrogen_masses	List of minimum nitrogen masses that the corresponding fertilizer applications should contain (kg).		0	
	phosphorus_masses	List of minimum phosphorus masses that the corresponding fertilizer applications should contain (kg).		0	
	potassium_masses	List of minimum potassium masses that the corresponding fertilizer applications should contain (kg).		0	
	application_depths	List of depths at which the fertilizer is injected into the soil (mm).		0	
	surface_remainder_fractions	List of fractions of fertilizer which remain on the soil surface when applied via injection.		0	1
	pattern_repeat	Number of times that this fertilizer application schedule should be repeated.	0	0	

Continued on next page

Table continued from previous page

Section	Variable	Definition	Default	Min	Max
	pattern_skip	Number of years to be skipped between schedule repetitions.	0	0	
	years	List of years in which tillage will occur.		1	
	days	List of days on which tilling will occur.		1	366
	tillage_depths	List of depths that the corresponding tillage applications reach (mm).		0.1	
	incorporation_fractions	List of fractions of surface pools that get incorporated into the soil during applications.		0	1
	mixing_fractions	List of fractions of soil layer pools that are available to mix into other soil layers.		0	1
	implements	List of tillage implements which will be used to execute the tillage operations.	-		
Manure Application					• subsoiler • moldboard-plow • coulter-chisel-plow • disk-harrow • cultivator • seedbed-conditioner
	pattern_repeat	Number of times that this tillage schedule should be repeated.	0	0	
	pattern_skip	Number of years to be skipped between schedule repetitions.	0	0	

Continued on next page

Table continued from previous page

Section	Variable	Definition	Default	Min	Max
	years	List of years in which manure will be applied.		1	
	days	List of days on which manure will be applied.		1	366
	nitrogen_masses	List of minimum nitrogen masses that the corresponding manure applications should contain (kg).		0	
	phosphorus_masses	List of minimum phosphorus masses that the corresponding manure applications should contain (kg).		0	
	potassium_masses	List of minimum potassium masses that the corresponding manure applications should contain (kg).		0	
	coverage_fractions	List of fractions of how much of the field is covered by the corresponding manure application.		0.01	1
	application_depths	List of depths at which the manure is injected into the soil (kg).		0	
	surface_remainder_fractions	List of fractions of manure which remain on the soil surface when applied.		0	1
	manure_types	The type of manure which will be requested for the application.		• liquid • solid	

Continued on next page

Table continued from previous page

Section	Variable	Definition	Default	Min	Max
	supplement_manure_nutrient_deficiencies	Determines if nutrient deficient manure applications are supplemented with chemical fertilizer.	none	• none • manure • synthetic fertilizer • synthetic fertilizer and manure	
	pattern_repeat	Number of times that this manure application schedule should be repeated.	0	0	
	pattern_skip	Number of years to be skipped between schedule repetitions.	0	0	

B. Methodology

The first step to initializing an instance of a field is to initialize an instance of the FieldData class for each field which is called by the setup_field_data() function in FieldManager. Each instance of the FieldData class contains key information about the field from the user-provided configuration data that remains fixed throughout the simulation as well as other field attributes that are updated dynamically throughout the simulation such as the amount of crop residue that is on the surface of the field (current_residue), the amount of water being applied to a field in an irrigation interval (watering_amount_in_mm) and the number of days since the start of the current watering interval (days_into_watering_interval).

Once the FieldData is initialized, the instances of the SoilData and SoilLayerData classes for that field are initialized. Following initialization of the soils for each field, the management schedules are generated. The user has some flexibility in how they provide information about the schedule for management operations so that they can either provide a list that provides information and dates for each operation individually, or if the same operation is repeated at an annual timestep or greater, they can provide information about the pattern with which that operation is repeated via the "pattern_repeat" and "pattern_skip" inputs. For example, if a user wanted to simulate an operation such as planting corn or fertilizer application that happened every year for 5 years on May 4th starting in 2021, they could specify this schedule with either of the two following combinations of inputs in a management schedule input file:

Option 1:

- "years": [2021, 2022, 2023, 2024, 2025],
- "days": [124, 124, 124, 124, 124],
- "pattern_repeat": 0,
- "pattern_skip": 0

Option 2:

- "years": [2021],
- "days": [124],
- "pattern_repeat": 5,
- "pattern_skip": 0

Similarly, if a user wanted to simulate something like cover crop planting or a manure application that happened every 4 years on September 5th starting in 1985, they could provide either of the following two inputs for that operation:

Option 1:

- "years": [1985, 1989, 1993, 1997, 2001, 2005],
- "days": [218]],
- "pattern_repeat": 0,
- "pattern_skip": 0

Option 2:

- "years": [1985],
- "days": [218],
- "pattern_repeat": 6,
- "pattern_skip": 4

To translate the user-provided information about field schedules into standard data formats, RuFaS generates management schedules via operation specific schedule classes. These schedule classes (CropSchedule, FertilizerSchedule, ManureSchedule, and TillageSchedule) are built as child classes from a generic parent class (Schedule) that contains functions to expand the user-provided information about the timing of management operations into a list with a Julian day and year for each management operation alongside any operation specific configuration data.

Crop specific management data includes specification of both the planting and harvesting schedule, and whether the harvest operation is a harvest only operation (meaning that the crop continues to grow after harvest) or if it is an event that kills the plant, with or without harvesting the crop.

Tillage specific management information includes the depth of the tillage operation, the amount of surface soil that is mixed into lower parts of the soil profile (incorporation_fractions), and the amount of mixing that occurs between soil layers (mixing_fractions).

Fertilizer application specific management information includes the proportions of N, P, and K in the fertilizer mixes being used, the proportion of N that is in the form of ammonium (ammonium_fraction), the total masses of N, P, and K applied at each application, the application depth, and the fraction of manure that remains on the surface (surface_remainder_fraction).

Manure application specific management information includes the total masses of N, P, and K applied at each manure application, the application depth, and the fraction of fertilizer that remains on the surface (surface_remainder_fraction), and an indication of how the model should handle cases where there is not enough manure in storage to meet the manure application request (supplement_manure_nutrient_deficiencies).

After the user information about the management schedules are translated into complete lists where each list entry is the schedule information for each operation for the entire simulation, these multi-year schedules are then translated into lists where each entry is an individual event for each operation so that the field manager can iterate over this list. The lists of events are then combined with the field and soil data to initialize an instance of a field!

Daily Field Management Each day the main simulation engine calls on the FieldManager routine to run its daily_update_routine() function which returns a list of harvested crops that can be passed to the Feed Storage Module. The FieldManager daily_update_routine() function collects the current weather conditions and manure for application to the field where appropriate and calls the Field class manage_field() routine which is the primary method in the Field class that steps through all the functions that execute the field management operations. The order of operations called is:

- Fertilizer Application
- Manure Application
- Tillage
- Soil Nutrient Cycling + Crop Growth
- Crop Management
  - Transition crops to dormancy
  - Plant crops
  - Harvest crops
  - Update crop and crop residue coverage

Once these daily processes have been executed for each field, the FieldManager daily_update_routine() function sends all the outputs to the OutputManager via the output_gatherer.send_daily_variables() function.

Fertilizer Application The first step in execution of a fertilizer application is to calculate the total mass of fertilizer required to meet the minimum N, P, and K application masses requested for each fertilizer given the specified nutrient composition of the fertilizer mix using the following two methods:

$$fert\_massnitrogen = \frac{mass\_requested\_nitrogen}{fert\_nitrogen\_fracfert\_mass}$$
$$phosphorus = \frac{mass_requested_phosphorus}{fert_phosphorus_frac}$$
$$fert\_masspotassium = \frac{mass\_requested\_potassium}{fert\_potssium\_frac}$$

[SC.FLD.1]

$$applied\_mass = max fert\_mass_{nitrogen},\ fert\_mass_{phosphorus}\ fert\_mass_{potassium}$$

[SC.FLD.2]

Where:

- applied_mass (kg) is the mass of the given fertilizer required to satisfy all nutrient requests.
- fert_massnitrogen (kg) is the mass nitrogen requested.
- fert_massphosphorus (kg) is the mass of phosphorus requested.
- fert_masspotassium (kg) is the mass of potassium requested.
- fert_nitrogen_frac (proportion) is the fraction of nitrogen in the fertilizer mix.
- fert_phosphorus_frac (proportion) is the fraction of phosphorus in the fertilizer mix.
- fert_potassium_frac (proportion) is the fraction of potassium in the fertilizer mix.

The mass of each nutrient applied is then calculated as:

$$applied\_nitrogen = applied\_mass * fert\_nitrogen\_frac$$
$$applied\_phosphorus = applied\_mass * fert\_phosphorus\_frac$$
$$applied\_potassium = applied\_mass * fert\_potassium\_frac$$

[SC.FLD.3]

Where:

- applied_nitrogen (kg) is the mass of nitrogen applied.
- applied_phosphorus (kg) is the mass of phosphorus applied.
- applied_potassium (kg) is the mass of potassium applied.

Once the total fertilizer mass and nutrients associated with a fertilizer application are determined, the fertilizer nitrogen and phosphorus masses are added to the appropriate soil nutrient pools via the apply_fertilizer() function in the FertilizerApplication class in the fertilizer_application.py file.

Note that RuFaS does not currently track soil potassium cycling so the application of potassium with fertilizer is recorded solely as a means of tracking resource use. In the future, we hope to add a soil potassium cycling routine at which point potassium will be added to soil nutrient pools via the apply_fertilizer() method in a similar manner as nitrogen and phosphorus.

Nitrogen and phosphorus are first added to the surface soil layer followed by distribution to the sub-surface layers according to the depth of application. Because soil nutrient pools are tracked in units of kg/ha, the total mass of each nutrient applied to the field is first divided by the field size.

The amount of each nutrient applied to the surface is determined as:

$$surface\_applied\_nutrient = \frac{(applied\_nutrient * surface\_remainder\_fraction)}{field\_size}$$

[SC.FLD.4]

Where:

- surface_applied_nutrient (kg/ha) is the mass per ha of each nutrient applied to the surface soil layer.
- surface_remainder_fraction (proportion) is the user-provided input defining the fraction of the fertilizer that remains on the soil surface.
- field_size (ha) is the size of the field.

The proportion of the subsurface fertilizer that is applied to each layer is calculated from the relationship between the bottom depth of each soil and the application depth. So, for each layer a “depth factor” is calculated as:

if layer_depth ¡ application_depth:

$$depth\_factor = \frac{layer\_depth}{application\_depth}$$

[SC.FLD.5]

else

$$depth\_factor = 1.0 - depth\_factor\_sum$$

[SC.FLD.6]

Where:

- depth_factor (proportion) is the fraction of subsurface fertilizer added to each soil layer
- layer_depth (mm) is the bottom depth of each sub-surface soil layer
- depth_factor_sum (proportion) is the sum of the depth factors for all layers above the deepest layer receiving fertilizer

Thus, the amount of fertilizer added to each nutrient pool in a sub-surface soil layer is:

$$sub\_surface\_applied\_nutrient = \frac{(applied\_nutrient * (1 - surface\_remainder\_fraction))}{field_size}$$

$$sub\_surface\_applied\_nutrient_{layer\_i} = sub\_surfaapplied\_nutrient * depth\_factor$$

The soil nutrient pools that receive nitrogen are the soil nitrate and ammonium pools. The proportion of fertilizer nitrogen distributed to each of these pools is determined by the user provided ammonium_fraction.

Fertilizer phosphorus is intended to be added to the labile inorganic phosphorus pools to be available for plant uptake. However, phosphorus applied to the surface is not immediately available in this pool and until the first precipitation or irrigation event the phosphorus is gradually sorbed by the soil. Thus, at the time of application, 75% of surface applied fertilizer P is added to the available phosphorus pool for gradual sorption and 25% of the surface applied P is added to the recalcitrant pool which is solubilized during rainfall and either added to the labile phosphorus pool or lost through runoff and leaching.

$$available\_phorphorus\_pool = surface\_applied\_phosphorus * 0.75$$
$$recalcitrant\_phorphorus\_pool = surface\_applied\_phosphorus * 0.25$$

[SC.FLD.7]

Manure Application When the results of the manure nutrient request are passed into the FieldManager and then to the Field class daily update routine, the manage_field() function can then execute the manure application with the manure nutrients and amount provided by the ManureModule via the SimulationEngine. If there are any unmet nutrients from the initial, intended application, if the user indicated that these unmet nutrients can be met with synthetic fertilizers, these are supplied by an additional fertilizer application that is initiated by the handle_unmet_nutrients() function.

Once the amount of manure to be applied is known, the apply_and_record_manure_application() function first adds the water associated with the manure application.The amount of water in the manure application is determined by the total mass of manure applied and the dry matter fraction of the manure:

$$manure\_applied\_water = \frac{manure\_mass\_applied}{manure\_dry\_matter\_fraction}$$

[SC.FLD.8]

Where:

- manure_applied_water (L) is the amount of water applied to the filed with a manure application

The amount of water in liters is then converted to the amount in mm by a helper function convert_liters_to_millimeters() .

After the water is applied, the phosphorus and nitrogen from manure are applied via the apply_machine_manure() function in the ManureApplication class which is implemented in manure_application.py. The manure application process is broken out into the surface application which follows methods described by the SurPhos model (Vadas et al., 2007) and the subsurface application.

Surface Manure Application During manure application, the amount of manure applied to the surface is assumed to stay on the surface unless it is a liquid manure with a dry matter content less than 15% in which case infiltration is assumed and the manure nutrients are assumed to be added to the sub-surface layer. Following the methods from the SurPhos model (Vadas et al., 2007), first the manure solids remaining on the soil surface are calculated:

$$surface\_dry\_matter\_mass = manure\_dry\_matter\_mass * surface\_remainder\_fraction$$

[SC.FLD.9]

Where:

- surface_dry_matter_mass (kg) is the amount of manure solids or dry matter that is applied to the soil surface at that application.
- manure_dry_matter_mass (kg) is the amount of manure solids or dry matter that is applied at that application.
- surface_remainder_fraction (kg) is the user-provided value for the amount of the manure applied that remains on the surface.

Surface Phosphorus Application Phosphorus is then distributed to 4 pools (water extractable inorganic phosphorus, water extractable organic phosphorus, stable inorganic phosphorus, stable organic phosphorus) according to the following methods:

First the fractions of each type of phosphorus are calculated assuming that 55% of manure phosphorus is water extractable and 75% of the non-water extractable or stable phosphorus is inorganic:

$$
\begin{gathered}
water\_extractable\_inorganic\_phosphorus\_fraction = 0.50 \\
water\_extractable\_organic\_phosphorus\_fraction = 0.50 \\
stable\_phosphorus\_fraction = 1 - (water\_extractable\_inorganic\_phosphorus\_fraction + \\
water\_extractable\_organic\_phosphorus\_fraction) \\
stable\_inorganic\_phosphorus\_fraction = 0.75 stable\_phosphorus\_fraction \\
stable\_organic\_phosphorus\_fraction = 0.25 stable\_phosphorus\_fraction
\end{gathered}
$$

[SC.FLD.10]

Then these fractions are multiplied by the total phosphorus applied as well as the product of the infiltration indicator and the surface remainder fraction to obtain the mass of each phosphorus type applied to add the surface or subsurface soil phosphorus layers:

$$
\begin{gathered}
mass\_to\_add\_to\_labile\_phos = \\
total\_phosphorus\_mass(water\_extractable\_inorganic\_phosphorus\_fraction + 0.95 * \\
water\_extractable\_organic\_phosphorus\_fraction + 0.95 * stable\_organic\_phosphorus\_fraction) * \\
soil\_infiltration * surface\_remainder\_fraction
\end{gathered}
$$

[SC.FLD.11]

$$
\begin{gathered}
mass\_to\_add\_to\_active\_phos = total\_phosphorus\_mass * stable\_organic\_phosphorus\_fraction * \\
soil\_infiltration * surface\_remainder\_fraction
\end{gathered}
$$

[SC.FLD.12]

Where:

- mass_to_add_to_labile_phos (kg) is the phosphorus that will be added to the surface labile phosphorus pool during that application.
- mass_to_add_to_active_phos (kg) is the phosphorus that will be added to the surface active phosphorus pool during that application.
- total_phosphorus_mass (kg) is the total phosphorus applied during that application.
- soil_infiltration is equal to 0.4 if the manure applied is less than 15% dry matter, and 0 otherwise.

The SurPhos manure pool attributes for the manure dry matter mass, moisture factor and field coverage are then updated according to the new field coverage and surface dry matter mass for that application (Vadas et al., 2007).

Surface Nitrogen Application The composition of manure nitrogen is provided by the manure module as the fractions that are in inorganic nitrogen, ammonium nitrogen, and organic nitrogen. These fractions are then distributed to the soil nitrate, ammonium, fresh organic nitrogen, and stable organic nitrogen pools according to the following:

$$mass\_nitrates\_added = manure\_dry\_matter\_mass * inorganic\_nitrogen\_fraction * \frac{(1 - ammonium\_fraction)}{field\_size}$$

$$mass\_ammonium\_added = manure\_dry\_matter\_mass * inorganic\_nitrogen\_fraction * \frac{(ammonium\_fraction)}{field\_size}$$

$$mass\_fresh\_organic\_nitrogen\_added = manure\_dry\_matter\_mass * organic\_nitrogen\_fraction * \frac{fresh\_fraction\_of\_organic\_nitrogen}{field\_size}$$

$$mass\_fresh\_organic\_nitrogen\_added = manure\_dry\_matter\_mass * organic\_nitrogen\_fraction * \frac{(1 - fresh\_fraction\_of\_organic\_nitrogen)}{field\_size}$$

[SC.FLD.13]

Where:

- mass_nitrates_added (kg N/ha) is the mass of nitrate nitrogen applied to relevant soil layer
- mass_ammonium_added (kg N/ha) is the mass of ammonium nitrogen applied to relevant soil layer
- mass_fresh_organic_nitrogten_added (kg N/ha) is the mass of fresh organic nitrogen applied to relevant soil layer
- mass_stable_organic_nitrogen_added (kg N/ha) is the mass of stable organic nitrogen applied to relevant soil layer
- manure_dry_matter_mass (kg) is the mass of manure dry matter added to the soil layer
- ammonium_fraction (proportion) is the fraction of inorganic nitrogen that is in the form of ammoniacal nitrogen and is provided by the Manure Module
- fresh_fraction_of_organic_nitrogen (proportion) is the fraction of organic nitrogen that is more quickly mineralized and is set to a constant of 0.9286

The manure mass that is applied to the surface layer is set to the 100% of the surface_dry_matter_mass calculated in **[SC.FLD.9]** if the manure dry matter content is greater than 15%. For liquid manure when the dry matter content is less than 15%, infiltration is assumed and the surface application is set to 60% of the surface_dry_matter_mass from **[SC.FLD.9]** and 40% of the surface_dry_matter_mass is assigned to be added to the first sub-surface layer.

Subsurface Manure Application For manure applications where a fraction or all of the manure is applied below the soil surface (i.e. injection) the sub-surface manure application method is followed for the proportion of the application applied below the soil surface.
The sub-surface fraction of manure application is simply calculated as:

$$subsurface\_fraction = 1 - surface\_remainder\_fraction$$

[SC.FLD.14]

to capture all the manure applied that is not accounted for in the surface application methods.This value, along with the manure nitrogen and phosphorus compositions and the total masses of manure applied in each application is passed into the apply_subsurface_manure() function.
Calculation of the manure phosphorus masses to be added to different soil phosphorus pools occurs according to the equations described in **[SC.FLD.9]**, **[SC.FLD.10]**, and **[SC.FLD.11]**.
Similarly, calculation of the manure N mass to be added to the subsurface layers follows the methods described in **[SC.FLD.13]**.
The method for distributing the total_phosphorus_mass and manure_mass_applied to the subsurface layers multiplies the manure application masses by the subsurface_fraction calculated in **[SC.FLD.9]** and depth_factors calculated using the method described in **[SC.FLD.5]** and **[SC.FLD.6]** as in the generic equation below:

$$nutrient\_mass\_added\_to\_layer = mass\_applied * nutrient\_fractionsubsurface\_fraction * depth\_factor$$

[SC.FLD.15]

Tillage Application The tillage method mixes nutrient masses from the soil layers to the layers below them and ensures that the nutrients do not get mixed below the bottom of the soil profile. The till_soil() function aggregates all the soil nutrient pools needed to be mixed through tillage including:

- ”labile_inorganic_phosphorus_content”,
- ”active_inorganic_phosphorus_content”,
- ”stable_inorganic_phosphorus_content”,
- ”nitrate_content”,
- ”ammonium_content”,
- ”active_organic_nitrogen_content”,
- ”stable_organic_nitrogen_content”,
- ”fresh_organic_nitrogen_content”,
- ”metabolic_litter_amount”,
- ”structural_litter_amount”,
- ”active_carbon_amount”,
- ”slow_carbon_amount”,
- ”passive_carbon_amount”

It then iterates through these soil nutrient pools and soil layers to redistribute nutrients between the soil pools according to the user provided tillage depth and mixing fraction.

Initiate Daily Biophysical Processes An important step in the Field classes manage_field() routine is to initiate the updates to the biophysical processes that are managed by the Soil and Crop classes. This is achieved by the execute_daily_processes() method and initiates the following processes:

- Snow accumulation and melting
- Crop residue and canopy cover update
- Soil temperature update
- Soil water and nutrient cycling updates
- Crop growth updates

Crop Management In managing the crop cycle, the daily updates called by the Field class first check if the crop is meant to go into a dormant stage.
After initiating any dormancy where appropriate, the Field class then checks to see if any crops are scheduled to be planted that day. If so, it calls on the Crop class to initialize Crop and ensures that the max_root_depth attribute of that crop does not allow its roots to go below the bottom of the soil profile.
The last management practice the Field class checks is the harvesting event schedule. If a harvest event is scheduled for that day, it initiates the harvest method in the Crop class and updates its list of crops to remove the harvested and killed crops from its list of crops.

IV. Soil Management

A. Introduction

The soil submodule spatially and dynamically models agronomic and biogeochemical processes taking place on the surface of and within the soil profile. It tracks plant roots/residue, hydrological movement, and dissolved/undissolved carbon, nitrogen, and phosphorus cycling. Carbon and nitrogen nutrient cycling and hydrology are based on the SWAT+ model and phosphorus nutrient cycling on the SurPhos model (Vadas et al., 2007).

Required Inputs

Table 2: Required inputs for the Soil section.

Variable	Definition	Default	Min	Max
second_moisture _condition_parameter	Curve number parameter for average moisture condition equation (unitless)	85	20	99
average_subbasin_slope	Slope of the field, measured as rise over run (unitless)	0.05	0	
slope_length	Slope of the field, measured as rise over run (m)	50	0	
manning_roughness_coefficient	Roughness coefficient for overland flow (unitless)	0.4	0.001	
support_practice_factor	Ratio of soil loss with a specific support practice based on slope characteristics. This value adjusts for terracing, contouring, and stripcropping, and these features are not present in V1, so this value IS NOT used (unitless)	0.08		
albedo	The ratio of reflected to total incident short-wave solar radiation. Related primarily to soil color and cover (unitless)	0.16	0	1
residue_fresh_organic _mineralization_rate	Fraction of residue that will decompose in a day given optimal conditions (unitless)	0.05	0	
soil_evaporation _compensation_coefficient	Modifies depth distribution used to meet soil evaporative demand (unitless)	0.95	0.01	1
initial_residue	Residual plant biomass remaining after a crop is harvested (kg h^{-1})	0	0	
soil_layers	The soil layers that this profile contains. Each element of the array should be a ‘soil_layer‘ object		1	
bottom_depth	The bottom depth of each soil layer object within the soil_layers array (mm)		20	500
wilting_point_water_concentration	Condition of soil moisture at which soil water becomes unavailable and water held in the soil at a tension of 1.5 Mpa (mm H_2O/mm soil)	0.2	0	0.95
field_capacity_water_concentration	Soil moisture condition at which the soil matric potential is zero or water held by the soil at 0.033 MPa (mm H_2O/mm soil)	0.3	0	0.95

Continued on next page

Table continued from previous page

Variable	Definition	Default	Min	Max
saturation_point_water _concentration	Concentration of water in a layer when it has become saturated (mm H_2O/mm soil)	0.5	0	0.95
saturated_hydraulic_conductivity	Measure of ease of water movement through the soil (mm hr-1)	9.5	0.01	15
bulk_density	Ratio of mass of solid particles to total volume of soil (g cm^{-3})	1.4	1.1	1.9
organic_carbon_fraction	Fraction of soil weight that is organic carbon at beginning of the simulation.	0.012	0	1
clay_fraction	Fraction of soil mass that is clay (unitless)	0.225	0	1
silt_fraction	Fraction of soil mass that is silt (unitless)	0.625	0	1
sand_fraction	Fraction of soil mass that is sand (unitless)	0.125	0	1
rock_fraction	Fraction of soil mass that is rock (unitless)	0.013	0	1
soil_water_concentration	Concentration of water in a soil layer at the beginning of the simulation (mm H_2O/mm soil)	0.25	0	0.95
pH	pH of the soil layer (pH)	7	0	14
initial_temperature	Temperature of the layer at the beginning of the simulation ($^{\circ}C$)	15.05		
initial_labile_inorganic _phosphorus_concentration	Concentration of labile inorganic phosphorus in the layer at the beginning of the simulation (mg kg^{-1} soil)	null	0	
initial_fresh_organic _phosphorus_concentration	Concentration of initial fresh organic phosphorus in the layer at the beginning of the simulation (mg kg^{-1} soil)	0	0	
initial_soil_nitrate_concentration	Concentration of nitrate in the layer at the beginning of the simulation (mg kg^{-1} soil)	null	0	
initial_soil_ammonium_concentration	Concentration of ammonium in the layer at the beginning of the simulation (mg kg^{-1} soil)	null	0	
humus_mineralization_rate_factor	Rate factor for humus mineralization of active organic nutrients (N and P) (unitless)	0.003	0	0.003
ammonium_volatilization _cation_exchange_factor	Controls the rate of ammonia volatilization (unitless)	0.45	0.01	

Continued on next page

Table continued from previous page

Variable	Definition	Default	Min	Max
denitrification_rate_coefficient	Controls the rate of denitrification (unitless)	1.4	0	3
denitrification_threshold_water_content	Fraction of field capacity water content above which denitrification takes place. (unitless)	1	0	1
residue_fresh_organic_mineralization_rate	The fraction of residue that will decompose in a day given optimal conditions (unitless)	0.05	0	

B. Methodology

Soil Temperature (Implemented in soil_temp.py)

Soil temperature fluctuates diurnally and seasonally and is estimated daily as a function of the previous day's soil temperature, average annual air temperature, the current day's soil surface temperature and depth in the soil profile for each soil layer by using the following equations from SWAT 2009 documentation.

Base equation

$$T_{soil}\ (z,\ d_n) = (L * T_{soil,(z,dn-1)}) + (1 - L) * [df * (T_{aair} - T_{surf}) + T_{surf}]$$

[SC.TMP.1]

Where:

- $T_{soil\ (z,d_n)}$ = soil temperature $(^{\circ}C)$ at depth z (mm) and day of the air d_n
- L = lag coefficient(ranging from 0.0 to 1.0, default: 0.8) that controls the influence of the previous day's temperature on the current day's temperature
- $T_{soil(z,d_n-1)}$ =soil temperature $(^{\circ}C)$ at depth z (mm) on previous day
- df = depth factor that quantifies the influence of depth below surface on soil temperature
- T_{aair} = Average annual air temperature $(^{\circ}C)$
- T_{surf} = Daily soil surface temperature $(^{\circ}C)$ on the day

$$df = \frac{zd}{zd + exp(-0.867 - 2.078 * zd})$$

[SC.TMP.2]

- zd = ratio of depth at the center of soil layer to damping depth

$$zd = \frac{z}{dd}$$

[SC.TMP.3]

Where:

- dd = damping depth (mm)
- z = depth at the center of the soil layer (mm)

Damping depth is a function of soil water and max damping depth. dd_{max} is the maximum damping depth (mm) to which the temperature wave penetrates and the amplitude decreases to 1/e or 0.37 of its initial value in the soil layers. This is largely dependent on soil thermal conductivity and thermal diffusivity, and is calculated using following equations

$$dd_{max} = 1000 + \frac{2500BD}{BD + 686exp(-5.63BD)}$$

[SC.TMP.4]

$$dd = dd_{max} * exp\{ln(\frac{500}{dd_{max}}) * [\frac{(1-scale)}{(1+scale)}]^2\}$$

[SC.TMP.5]

- dd_{max} = maximum damping depth (mm)
- scale = scaling factor for soil water

$$scale = \frac{SW}{[(0.356-0.144BD)*Ztot]}$$

[SC.TMP.6]

Where:

- BD = soil bulk density (g/cm^3)
- SW = total soil water in the profile (mm)
- Ztot = total soil profile depth (mm)

Soil surface temperature is a function of the previous day's temperature, the amount of ground cover, and the temperature of a bare soil surface. The temperature of a bare soil is calculated as:

$$T_{bare} = T_{av} + E_{sr}^{\frac{(Tmax-Tmin)}{2}}$$

[SC.TMP.7]

Where:

- T_{bare} = Temperature of a bare soil surface ($^\circ C$) with no cover
- T_{av} = Average daily temperature ($^\circ C$) on the day
- E_{sr} = radiation term

$$E_{sr} = \frac{[Hday*(1-albedo)-14]}{20}$$

[SC.TMP.8]

Where:

- H_{day} = daily solar radiation (user input, MJ/m^2/d)
- albedo = daily fraction of incoming sunlight that is reflected by a surface

Albedo should be determined by soil type and is set by user input.
The impact of plant and snow cover is quantified as a weighting factor, bcv, given by the maximum of the following two equations:

$$bcv = \frac{plant\_cover}{plant\_cove + exp(7.563 - 0.0001297) * plant\_cove}$$

[SC.TMP.9]

$$bcv = \frac{SNOW}{SNOW + exp(6.055 - 0.3002) * SNOW}$$

[SC.TMP.10]

Where:

- SNOW = snow water content on the current day (mm)
- plant_cove = Total aboveground plant biomass and residue on the current day (kg per hectare)

$$Tsurf = (bcv * T_{soil,d-1}) + [(1 - bcv) * T_{bare}]$$

[SC.TMP.11]

- Tsurf = surface temperature ($^{\circ}C$)

Surface temperature is dependent on the previous day's soil temperature and is calculated before soil temperature on any given day. On the first day of a simulation the default soil temperature is used for $T_{soil,d-1}$.

Soil Hydrology

Infiltration (implemented in infiltration.py) The movement of soil water into and through the soil profile. When application of water to the ground surface exceeds the rate of infiltration, it results in surface runoff. Soil texture (percentage of sand, silt, and clay), bulk density, presence of pores, pore diameter, and continuity affect the soil infiltration rate. Slow soil infiltration rate results in ponding in the level areas, surface runoff, and erosion in sloping areas. High infiltration rates can lead to nutrient leaching. Surface runoff is estimated using the NRCS curve number method.
Runoff has a minimum of 0 and is calculated as:

$$if R > 0.2S,\ runoff = R - 0.2S2R + 0.8S$$
$$if\ not,\ runoff = 0$$

[SC.INF.1]

Where:

- R = daily rainfall (mm H_2O)

- S = retention parameter (mm H_2O)

S is determined using an empirical value CN (curve number, unitless, user-defined). Three values of CN are used to determine S.

$$CN_1 = CN_2 - \frac{20*100-CN_2}{100-CN_2+exp[2.533-0.0636*(100-CN_2)]}$$

[SC.INF.2]

$$CN_3 = CN_2 * exp[0.00673 * (100 - CN_2)]$$

[SC.INF.3]

Where:

- CN_1, the first moisture condition, is the wilting point.
- CN_2, the second moisture condition, curve number is user-defined.
- CN_3, the third moisture condition, is field capacity.

S is calculated as

$$S = S_{max} * \{1 - \frac{SW_{available}}{SW_{available}+exp[w_1-(w_2*SW_{available})]}\}$$

[SC.INF.4]

- S_{max} = maximum value for S on any given day (mm H_2O)
- $SW_{available}$ = available soil profile water content (SW - WP) (mm H_2O)
- $w_1,\ w_2$ = shape coefficients

Smax is calculated as:

$$S_{max} = 25.4 * [\frac{1000}{CN_1} - 10]$$

[SC.INF.5]

w1 and w2 are calculated as:

$$w_1 = ln\{\frac{FC}{1-(S_3*S_{max}^{-1})} - FC\} + w_2 * FC$$

[SC.INF.6]

$$w_2 = \frac{ln\{\frac{FC}{1-(S_3 * S_{max}^{-1})} - FC\} - ln\{\frac{SAT}{1-(2.54*S_{max}-1)} - SAT\}}{SAT - FC}$$

[SC.INF.7]

Where:

- FC = amount of water in soil profile at field capacity (mm H_2O)
- SAT = amount of water in soil profile at saturation (mm H_2O)

S3 is calculated as:

$$S_3 = 25.4 * [\frac{1000}{CN_3} - 10]$$

[SC.INF.8]

When the top layer of the soil is frozen (layerTemp $\leq 2°C$), S is modified using the following equation:

$$S_{frz} = S_{max} * [1 - exp(-0.000862 * S)]$$

[SC.INF.9]

Infiltration into the soil is daily rainfall less daily runoff.

$$Infiltration = R - runoff$$

[SC.INF.10]

Evapotranspiration Evapotranspiration (ET) is one of the most important hydrological processes of soil water balance in the soil and crop module. It is the sum of all processes by which water moves from the land surface to the atmosphere via soil evaporation, plant canopy transpiration, and sublimation. The Hargreaves method requires air temperature only. RuFaS adapts the Hargreaves method (Hargreaves and Samani, 1985) to estimate ET which only requires temperature as an input.

Potential evapotranspiration (PET) is the maximum amount of water that could be removed due to surface evaporation and plant canopy transpiration with no limitation on water availability, while evapotranspiration (ET) is the actual amount of water lost due to evaporation and transpiration.

Potential Evapotranspiration (Implemented in field.py)

$$LHV * ET_{max} = 0.0023 * H_0 * (T_{max} - T_{min})^{0.5} * (T_{avg} + 17.8)$$

[SC.EVP.1]

Where:

- LHV = latent heat of vaporization (MJ kg^{-1})
- ET_{max} = potential evapotranspiration (otherwise known as PET) (mm d^{-1})
- H_0 = extraterrestrial radiation (MJ m^{-2} d^{-1}), input
- T_{max} = maximum air temperature for a given day ($^\circ C$)
- T_{min} = minimum air temperature for a given day ($^\circ C$)
- T_{avg} = mean air temperature for a given day ($^\circ C$)
- ET_{max} is taken as the maximum between 0.001 and the value calculated by solving for ET_{max} in the equation above.

LHV is calculated as:

$$LHV = 2.501 - 2.361 * 10^{-3} * T_{avg}$$

[SC.EVP.2]

Maximum crop transpiration (implemented in water_dynamics.py)

$$\begin{aligned} trans_{max} &= \frac{ET_{max} * LAI_{act}}{3.0} && if 0 \leq LAI_{act} \leq 3.0 \\ trans_{max} &= ET_{max} && if LAI_{act} > 3.0 \end{aligned}$$

[SC.EVP.3]

Where:

- $Trans_{max}$ = maximum transpiration on a given day (mm H_2O)
- LAI_{act} = LAI (Leaf Area Index) actual (calculated in Crop Growth)

LAI lower than 3.0 limits evapotranspiration. Actual transpiration values may be less than the calculated maximum amount due to lack of available water in the rooting depth of the soil profile.

Sublimation and soil evaporation (implemented in field.py)

$$evap_{max} = ET_{max} * SoilCov$$

[SC.EVP.4]

Where:

- $evap_{max}$= maximum soil evaporation/sublimation on a given day (mm H_2O)
- SoilCov = soil cover index

$$SoilCov = exp(-5.0 * 10^{-5} * BioMass)$$

[SC.EVP.5]

Where:

- BioMass = aboveground biomass and residue (kg ha^{-1})

Potential evaporation is adjusted based on $trans_{max}$ as:

$$evap_{max} = min\left\{evapmaxevapmaxETmaxevapmax + transmax\right\}$$

$$evap_{\text{max}} = min\left\{\frac{evap_{\text{max}} \cdot ET_{\text{max}}}{evap_{\text{max}} + trans_{\text{max}}},\ evap_{\text{max}}\right\}$$

[SC.EVP.6]

Partition evaporation within soil profile (implemented in evaporation.py)

$$evap_z = evap_{max} * [\frac{z}{z+exp(2.374-0.00713*z)}]$$

[SC.EVP.7]

Where:

- $evap_z$ = evaporation demand at depth z (mm H_2O)
- z = depth below soil surface (mm)

The evaporation demand for a given soil layer is the difference between evaporation demands at the top and bottom of the layer.

$$evap_{ly} = evap_{bottom} - evap_{top}$$

[SC.EVP.8]

When the water content of a soil layer is below field capacity, the evaporative demand for the layer is reduced according to the following equation: evaporation.py

$$evap_{ly} = evap_{ly} * exp[2.5 * \frac{SW-FC}{FC-WP}] \qquad if\ SW < FC$$

[SC.EVP.9]

Where:

- SW = soil water content for the layer (mm H_2O)
- WP = soil water content held at wilting point (mm H_2O)
- FC = field capacity for soil water (mm H_2O)

In addition, the daily amount of water removed by evaporation is limited to 80% of the plant available water (SW-WP): evaporation.py

$$evap_{ly} = min = min\left\{0.8 * SW - WP,\ evap_{ly}\right\}$$

[SC.EVP.10]

Evaporation for each layer is added to ETact, the total actual amount of water lost in evapotranspiration.

Percolation (implemented in percolation.py) Percolation is the process of water movement downward through the soil profile. When the soil layer water is less than or equal to field capacity no percolation occurs. Percolation is not calculated for frozen layers.

$$perc = SW_{perc} * [1 - exp(\frac{-t}{TT})]$$

[SC.PER.1]

Where:

- SWperc = water available for percolation (mm H_2O)
- perc = amount of water that percolates to the underlying soil layer (mm H_2O)
- t = time step (24 h)
- TT = travel time for percolation (h)

$$SW_{perc} = SW - FC \qquad if SW > FC$$

[SC.PER.2]

TT for each soil layer is calculated as:

$$TT = \frac{SAT - FC}{Ksat}$$

[SC.PER.3]

Where:

- Ksat = saturated hydraulic conductivity (mm/h)
- SAT = Amount of water in the soil layer when completely saturated (mm H_2O)

Soil Erosion (Implemented as soil_erosion.py)

Base Equation, Rainfall Intensity, and Peak Runoff

Soil erosion is a geological process whereby surface soil is removed by water/wind and anthropogenically. In RuFaS, erosion is modeled from individual crop fields using the Modified Universal Soil Loss Equations (MUSLE, SWAT 2009 documentation). The MUSLE input variables include rainfall (rainfall amount and intensity), soil erodibility factor, soil cover/management, best management practices and the topographic factor.

C. Base Equation, Rainfall Intensity, and Peak Runoff

$$sed = 11.8 * (Qsurf * qpeak * field\_size)^{0.56} * K * C * P * LS$$

[SC.ERO.1]

Where:

- sed = sediment yield on a given day (metric tons)
- Qsurf = surface runoff volume (m^3)
- qpeak = peak runoff rate (m^3/sec)
- field_size = Area of the cropping field (ha)
- K = USLE soil erodibility factor (Mg $MJm^{-1}mmm^{-1}$))
- C = USLE cover and management factor
- P = USLE support practice factor
- LS = USLE topographic factor

q_{peak} is calculated as:

$$qpeak = \frac{RC * I * Area}{3.6}$$

[SC.ERO.2]

Where:

- RC = runoff coefficient
- I = rainfall intensity (mm/hr)
- Area = field size (ha)

RC is the fraction of daily runoff to daily precipitation

$$RC = \frac{runoff}{R}$$

[SC.ERO.3]

Where:

- runoff = runoff (mm H_2O)
- R = rainfall (mm H_2O)

Rainfall intensity is rainfall over time for the time of concentration

$$I = \frac{Rtc}{Tconc}$$

[SC.ERO.4]

Where:

- Rtc = rain amount during time of concentration (mm H_2O)
- Tconc = time of concentration (h)

$$Tconc = \frac{Length^{0.6} * n^{0.6}}{18 slope^{0.3}}$$

[SC.ERO.5]

Where:

- Length = slope length of field (m)
- n = manning's roughness coefficient (Refer Table 2)
- slope = field slope (m/m)

$$Rtc = alpha * R$$

[SC.ERO.6]

Where:

- alpha = fraction of daily rain during time of concentration

$$alpha = 1 - exp[2 * Tconc * ln(1 - alpha_{0.5})]$$

[SC.ERO.7]

Where:

- $alpha_{0.5}$ = fraction of daily rain in in ½ hour of highest intensity

$$alpha_{0.5} = \frac{\{0.02083 + [1 - exp(\frac{-125}{R+5})]\}}{2}$$

[SC.ERO.8]

Soil erodibility factor K is calculated as:

$$K = f_{csand} * f_{cl-si} * f_{orgc} * f_{sand}$$

[SC.ERO.9]

Where:

- Fcsand = gives low factors for soils with high sand contents and high values for soils with little sand
- Fcl-si = gives low factors for soils with high clay to silt ratios
- Forgc = reduces soil erodibility for soils with high organic carbon content
- Fsand = reduces soil erodibility for soils with high sand contents

Factors are calculated as:

$$f_{csand} = 0.2 + 0.3 * exp[-0.256 * sand * (1 - \frac{silt}{100})]$$

[SC.ERO.10]

$$f_{cl-si} = [\frac{silt}{clay+silt}]^{0.3}$$

[SC.ERO.11]

$$f_{orgc} = 1 - \frac{0.25orgC}{orgC+exp(3.72-2.95*orgC)}$$

[SC.ERO.12]

Where:

- orgC = organic Carbon percentage for the first soil layer.

$$f_{sand} = 1 - \frac{0.7*(1-\frac{sand}{100}}{(1-\frac{sand}{100})+exp(-5.51+22.9*(\frac{sand}{100})^{-1})}$$

[SC.ERO.13]

The cover and management factor, C, is the ratio of soil loss from land cropped under specified conditions to loss from clean-tilled, continuous fallow. The equation below essentially allows C to vary between 0.8 and 0.05 as:

$$C = exp\{[ln(0.8) - ln(0.05)] * exp(-0.00115 * surface\ residue) + ln(0.05)\}$$

[SC.ERO.14]

Where:

- Cover = amount of residue and growing biomass covering the soil surface (kg/ha)
- 0.05 = the estimated minimum value for C

LS is a topographic factor and is the expected ratio of soil loss per unit area from a field slope to that from a 22.1-m length of uniform 9 percent slope under identical conditions. LS is estimated as:

$$LS = \frac{Lhill}{22.1}^{m} * [65.41 * sin^2(alphahill) + 4.56 * sin(alphahill) + 0.065]$$

[SC.ERO.15]

Where:

- Lhill = hill slope length (user input, m)
- Alphahill = angle of the hill slope, defined as $\tan^{-1}$(average subasin slope) (user input, m/m)
- average subbasin slope = average slope for the hydrological subbasin (user input, m/m)

The exponent m is calculated as

$$m = 0.6 * [1 - exp(-35.835 * average\ subbasin\ slope)]$$

[SC.ERO.16]

The P factor is a user defined support practice factor that reduces erosion.

If there is snow on the ground, this will heavily affect sediment yield. sed is adjusted for snowfall on the range day $>$ 350, or day $<$ 60 as:

$$sed_{snow} = \frac{sed}{exp(\frac{3 * snow\ water\ content}{25.4})}$$

[SC.ERO.17]

Where:

- sedsnow = snow adjusted sediment yield.

Coarse Fragment factor (CFRG) is calculated as :

$$CFRG = exp(-0.053 * rock)$$

[SC.ERO.18]

Where:

- rock is the percent of rock in the first soil layer

Soil Nitrogen

RuFaS tracks soil N as ammonium, nitrate, and five pools based on nitrogen form and lability. Three pools are organic N; fresh, active, and stable. Two are inorganic N; NH_4^+ and NO_3^-. Nitrogen enters the cycle as synthetic fertilizer, manure, and crop residue. Synthetic fertilizers are partitioned into nitrate and ammonium. Manure organic N is partitioned into active and stable organic N pools by 92.86% and 7.14% respectively. Active and stable organic N can interconvert. N in the active pool can also get percolated to the second layer. Furthermore, manure dry matter mass goes into ammonium and nitrate pools using certain equations. Residue biomass also gets converted into fresh and active organic N pools via mineralization. If residue is in top 2 cm it goes into the fresh organic N pool while if residue is below 2 cm it goes into the active organic N pool. 80% of fresh organic N is converted into nitrate and 20% into active pool via decomposition. Active N is converted into NH_4^+ after mineralization. NH_4^+ can be converted into NO_3^- via nitrification, taken up by plants, volatilized, percolated, or lost via runoff. NO_3^- can be taken up by plants, percolate, runoff, or denitrify to NO, N_2O, and N_2 gas.

Initialize soil N Pools (Implemented in layer_data.py)

$$initial\_soil\_nitrate\_concentration = 7 * e^{\frac{-depth\_of\_layer\_center}{1000}}$$

[SC.SON.1]

Where:

- initial_soil_nitrate_concentration is the initial concentration of nitrate in the soil (mg kg^{-1} or ppm) at depth.
- depth_of_layer_center is the depth of the middle of the soil layer from the soil surface (mm).

The ammonium pool for soil N is initially set by the user in the soil input file.

Organic N values in the soil profile are set by the following equation:

$$humic\_organic\_nitrogen\_concentration = 10^4 * \frac{organic\_carbon\_fraction * 100}{14}$$

[SC.SON.2]

Where:

- humic_organic_nitrogen_concentration is the concentration of humic organic N in the layer (mg kg^{-1} or ppm).
- organic_carbon_fraction is the percent organic C in the layer (%, user input).

Humic organic N is partitioned between the active and stable pools for each layer using the following equations:

$$activeN = humic\_organic\_nitrogen\_concentration * FractN$$
$$stableN = humic\_organic\_nitrogen\_concentration * (1 - FractN)$$

[SC.SON.3]

Where:

- activeN, initial_active_organic_nitrogen_concentration, is the concentration of N in the active organic pool (mg kg^{-1}).
- humic_organic_nitrogen_concentration is the concentration of humic organic N in the layer (mg kg^{-1}).
- FractN, fraction_of_humic_nitrogen_in_active_pool, is the fraction of humic N in the active pool; General Constant - 0.02.
- stableN, initial_active_organic_nitrogen_concentration, is the concentration of N in the stable organic pool (mg kg^{-1}).

Fresh N is initialized in the top soil layer only (user defined, usually 0-20 mm), and is 0.15% of the initial amount of residue on the soil surface.

$$fresh\_organic\_nitrogen\_content = 0.0015 * residue$$

[SC.SON.4]

Where:

- fresh_organic_nitrogen_content is the fresh organic pool in the top soil layer (kg N ha^{-1}).
- residue is material in the residue pool for the top soil layer (kg ha^{-1}).

While pool sizes are initialized as concentration (mg kg^{-1}), all calculations are done in units of mass (kg ha^{-1}). The conversion is:

$$mass(kgNha^{-1}) = \frac{(nutrientconc(mgkg^{-1}) * BD * depth * fieldsize)}{fieldsize}$$

[SC.SON.5]

$$mass(kgNha^{-1}) = mgkg^{-1} * BD * depth10$$

[SC.SON.6]

Where:

- BD is the bulk density for the given layer (g cm^{-3} or mg m^{-3}).
- depth is the thickness of a given layer (mm).
- field size is the field area (ha).

All pools have a lower limit of 0. If a transfer of N from one pool to another would is greater than the N in the pool of origin, the destination pool receives only the amount that depletes the pool of origin. The rest of the N cycle is in the nitrogen_cycling folder and run by nitrogen_cycling.py, which is called from the soil routine.

Nitrification and Volatilization (Implemented in nitirification_volatilization.py) *Nitrification* Nitrification is conversion of NH_4^+ to NO_3^- via microbial oxidation of NH_4^+ under suitable soil temperature and moisture conditions. Nitrification occurs only at soil temperatures greater than 5°C and is modeled as a function of soil temperature and water factors, temp_factor and water_factor.

$$temp\_factor = min(1.0,\ 0.41\ X \frac{temperature-5}{10})\ if\ temperature > 5$$

[SC.NNV.1]

Where:

- temperature is the temperature in the soil layer (°C).

If:

$$water\_content \geq 0.25 * field\_capacity - 0.75 * wilting\_pointwater\_factor = 1$$

Else:

$$water\_factor = \frac{water\_content - wilting\_point}{0.25 * (field\_capacity - wilting\_point)}$$

Where:

- water_content is the water content of the soil layer on a given day (mm).
- field_capacity is the field capacity water content (mm).
- wilting_point is the wilting point water content (mm).

Volatilization The volatilization of aqueous soil ammonium to ammonia gas (NH_3) occurs even when nitrification does not. NH_3 volatilization is influenced by temperature, moisture, fertilizer application, pH, and depth (University of Missouri Extension, 2022).

The volatilization depth factor (DepthFac) is unitless and calculated as:

$$depth\_factor = 1 - \{\frac{depth\_of\_layer\_center}{depth\_of\_layer\_center + exp(4.706 - 0.0305 * depth\_of\_layer\_center)}\}$$

[SC.NNV.2]

Where:

- depth_of_layer_center is the depth from the surface of the soil to the middle of the layer (mm).

Nitrification and Volatilization Regulation The impact of environmental factors on nitrification and NH_3 volatilization are estimated by nitrification and volatilization regulator equations. Nitrification and volatilization only occur if the soil temperature is greater than 5°C.

A nitrification regulator (nitrification_regulator) and a volatilization regulator (volatilization_regulator) are calculated as:

$$nitrification\_regulator = temp\_factor * water\_factor$$
$$volatilization\_regulator = temp\_factor * depth\_factor * cation\_exchange\_factor$$

[SC.NNV.3]

Where:

- cation_exchange_factor is provided by the soil input file..

The total amount of NH_4 (kg N ha^{-1}) converted through nitrification and volatilization is given by:

$$total\_ammonium\_lost = NH_4 * [1 - exp(-1 * nitrification\_regulator - volatilization\_regulator)]$$

[SC.NNV.4]

The estimated fraction of total_ammonium_lost that is converted to NO_3 by nitrification is:

$$nitrification\_loss\_fraction = 1 - exp(-1 * nitrification\_regulator)$$

[SC.NNV.5]

The estimated fraction of total_ammonium_lost that is lost by volatilization (FracVolatil) is:

$$volitilization\_loss\_fraction = 1 - exp(-1 * volitilization\_regulator)$$

[SC.NNV.6]

The amount of N removed from the NH_4 pool via nitrification and converted to NO_3 is (kg N ha^{-1}) calculated as:

$$nitrification\_loss = [\frac{nitrification\_loss\_fraction}{nitrification\_loss\_fraction + volitilization\_loss\_fraction}] * total\_ammonium\_lost$$

[SC.NNV.7]

If:

$$nitrification\_loss\_fraction + volitilization\_loss\_fraction \leq 0$$
$$nitrification\_loss\ is\ 0$$

N loss in leaching and runoff (Implemented in leaching_runoff_erosion.py) N can be transported with moving water vertically through the soil profile or into streams and water bodies via leaching, runoff, and erosion. Leaching is the process where N ions move down the soil profile with percolating soil water (Wang and Li, 2019). Runoff and erosion N is removed exclusively from the surface soil layer. Lateral flow is not accounted for.

The concentration of NO_3 and NH_4 (kg N mm^{-1} H_20) in the mobile water fraction for a given layer ($NConc_1$) is:

$$mobile\_water\_nitrogen\_concentration = \frac{NO_3 \ or \ NH_4 * [1 - exp(\frac{-total\_mobile\_water}{(1-\theta)*SAT})]}{total\_mobile\_water}$$

[SC.NNV.8]

Where:

- NO_3 or NH_4 is the amount of nitrate or ammonium in the layer (kg N ha^{-1})
- total_mobile_water is the amount of mobile water in the layer (mm H_2O)
- θ is the fraction of porosity from which anions are excluded
- SAT is the saturated water content of the soil layer (mm H_2O).

The amount of mobile water in the layer (mm H_2O) is the amount of water lost by surface runoff and percolation given as:

for top layer: $$total\_mobile\_water = runoff\_water\_amount + percolated\_water\_amount$$

[SC.NNV.9]

for lower soil layers: $$total\_mobile\_water = percolated\_water\_amount$$

[SC.NNV.10]

Where:

- runoff_water_amount is the surface runoff generated on a given day (mm H_2O)
- percolated_water_amount is the amount of water percolating to the underlying soil layer on a given day (mm H_2O).

The amount/mass (kg N ha^{-1}) of NO_3 and NH_4 lost in surface runoff is:

$$nitrates\_lost\_to\_runoff = nitrate\_conc\_in\_mobile\_h20 * nitrate\_runoff\_coefficient * runoff\_water\_amount$$

$$ammonium\_lost\_to\_runoff = ammonium\_conc\_in\_mobile\_h20 * ammonium\_runoff\_coefficient * runoff\_water\_amount$$

[SC.NNV.11]

Where:

- xxx_conc_in_mobile_h20 is the concentration of nitrate or ammonium in the surface layer mobile water (kg N mm^{-1} H_2O)
- xxx_runoff_coefficient is the nitrate or ammonium percolation coefficient (GeneralConstants; ammonium = 0.2, nitrate = 0.2)
- runoff_water_amount is the surface runoff generated on a given day (mm H_2O)

N erosion (Implemented in leaching_runoff_erosion.py) Soil N is eroded from the organic N pools; fresh, active, and stable.

$$NConc = \frac{100 * N(kgN/ha)}{bulk\_density * depth}$$

[SC.LCH.1]

Where:

- bulk_density is the soil layer bulk density (Mg m^{-3})
- depth is the thickness of the soil layer (mm).

Organic N mass loss in erosion (kg N ha^{-1}) is calculated as:

$$erosion\_nitrogen\_loss = 0.001 * nitrogen\_erosion\_concentration * daily\_soil\_lost * enrichment\_ratio$$

[SC.LCH.3]

Where:

- daily_soil_lost is the daily soil loss per hectare (metric tons soil ha^{-1})
- nitrogen_erosion_concentration is the concentration of organic N in the top soil layer (g N metric ton^{-1} soil)
- enrichment_ratio is the ratio of concentration of organic N transported with the sediment to that found in the soil surface layer.

$$enrichment\_ratio = exp[1.21 - 0.16 * log(daily\_soil\_lost * 1000)]$$

[SC.LCH.4]

Notes:

- Leaching affects the inorganic and active organic pools. If percolation is 0, then percolated N is also 0. NO_3, NH_4 and active N lost to percolation in each layer is removed from the layer of origin and added to the layer below. Water and N percolated from the bottom layer are lost from the soil profile into the vadose layer.

$$Percolated\_ammonium = ammonium\_conc\_in\_mobile\_h20 * percolated\_water$$
$$Percolated\_nitrate = nitrate\_conc\_in\_mobile\_h20 * percolated\_water$$

[SC.LCH.5]

Where:

- xxx_conc_in_mobile_h20 is the concentration of nitrate or ammonium in the surface layer mobile water (kg N mm^{-1} H_2O) as determined in equation **[SC.NNV.8]**
- Percolated_water is the water percolated from the layer (mm)

Denitrification (Implemented in denitrification.py) Denitrification is the bacterial conversion of NO_3 to denitrified nitrates (N_2, NO_2, NO, N_2O) under anaerobic conditions. The denitrification submodule calculates total denitrified nitrates and partitions these emissions further into N_2O and N_2. Denitrification is a function of soil water content, temperature, and induced by the presence of NO_3 and carbon (C).

The amount of nitrate that is lost to denitrification (DenitrN) (kg N ha^{-1}) is calculated for each soil layer by the function calculate_denitrification_amount according to the following equation:

If nutrient_cycling_soil_water_factor$_{layer[i]}$ $>$ soil_water_threshold:

$$denitrified\_nitrates_{layer[i]} = nitrate\_content_{layer[i]} * max(min(1.0, 1 - exp(-denitrification\_rate\_coefficient * temp\_factor_{layer[i]} * organic\_carbon\_percent_{layer[i]})),$$

[SC.NDN.1]

Else:

- denitrified_nitrates$_{layer[i]}$ = 0

Where:

- nitrate_content$_{layer[i]}$ is the amount of nitrate in layer i (kg N h^{-1})
- denitrification_rate_coefficienct is the user defined denitrification rate coefficient which can have values between 0-3 with a default value of 1.4
- temp_factor$_{layer[i]}$ is the nutrient cycling temperature factor for each layer,
- organic_carbon_percent$_{layer[i]}$ is the amount of organic carbon in the layer (%) calculated as organic_carbon_fraction$_{layer[i]}$ * 100
- nutrient_cycling_soil_water_factorlayer[i] is the nutrient cycling water factor for the layer i

- soil_water_threshold is the nutrient cycling water factor threshold value for the layer. According to Wen et al. (2024) the value for the SWthr is typically set to 1.0 meaning that denitrification is allowed to occur when water content is above field capacity.

The denitrified_nitrates$_{layer[i]}$ is removed from the layer's NO_3 pool and partitioned into N_2O and N_2 which are then lost to the atmosphere.

Partitioning factor is used to partition denitrified nitrates into N_2O and N_2

$$Partitioning\_factor = max((min\ (NO3\_effect, C\_effect) * moisture\_effect * pH\_effect), 0.0)$$

[SC.NDN.2]

$$NO3\_effect = (1.0 - [0.5 + \{atan(pi * 0.01 * \frac{(NO3\_content - 190)}{pi})\}] * 25)$$

[SC.NDN.3]

$$Ceffect = 13 + \{\frac{[30.78 * atan[pi * 0.07 * (Cresp * 1000 - 13)]}{pi}\}$$

[SC.NDN.4]

$$moisture\_effect = \frac{1.4}{(13^{17/(13^{(2.2*WFPS)})})}\ if\ WFPS = 0,\ moisture\ effect = 0$$

[SC.NDN.5]

$$pH\_effect = \frac{1}{(1470 * e^{(-1.1*pH)})}$$

[SC.NDN.6]

Where:

- NO3_effect is the effect of nitrate on partitioning (unitless)
- C_effect is the effect of carbon on partitioning (unitless)
- atan= the arc tangents measured in radians of x (values are between -pi/2 and pi/2)
- NO3_content is the amount of nitrate in the layer (ug N/g soil)
- Cresp is carbon respiration from the soil layer (kg ha^{-1})
- WFPS is the water filled pore space in the soil layer (unitless).

The amount of N_2O-N emissions (kg N ha^{-1}) are calculated as follows:

$$nitrous\_oxide\_nitrogen = (\frac{denitrified\_nitrates}{(1+partition_factor)}) * 0.001$$

[SC.NDN.7]

$$N_2O(kg) = (\frac{nitrates}{(1+partition_factor)}) * 0.001$$

[SC.NDN.8]

Where:

- denitrified_nitrates is the amount of nitrates that have been denitrified (kg ha^{-1})

Mineralization and Decomposition/Immobilization (Implemented in mineralization_decomp.py)

Mineralization is the microbial conversion of organic N to simple, soluble forms of organic N (including amino acids) and ultimately to the inorganic N that can be taken up by a plant. Decomposition is the breakdown of fresh organic residue into simpler organic components. Immobilization is the uptake or assimilation of N by microbes making them unavailable (temporarily) to plants. Soil stoichiometry (C:N) determines whether organic N will be mineralized (resulting in more plant available N) or inorganic N will be immobilized (resulting in less plant available N).

The N Mineralization equations represent net mineralization which incorporate immobilization into the equations. Fresh and active N pools are subject to decomposition and mineralization respectively. Mineralization and decomposition are allowed to occur only if soil temperature is greater than 0$^{\circ}C$. Soil temperature and water are the two factors used in the mineralization and decomposition equations to account for the impacts of temperature and moisture on these processes.

N mineralized from the active N pool (Nminact) (kg N ha^{-1}) is calculated as:

$$Nminact = Minrate * (temp\_factor * water\_factor)^{0.5} * activeN$$

[SC.NMN.1]

Where:

- Minrate is the rate coefficient for mineralization of the humus active organic nitrogen
- TempFac is the nutrient cycling temperature factor for the layer
- WaterFac is the nutrient cycling water factor for the layer
- ActiveN is the amount of N in the active organic pool (kg N ha^{-1}).

N mineralized from the active pool is transferred from the active pool to the NH_4 pool. Decomposition and mineralization are a function of a daily decay rate constant that is calculated with the C:N and C:P ratios of the residue, and temperature and soil water factors. The C:N and C:P ratios of the residues in the soil layer are calculated as:

$$C:N = \frac{structural\ litter + metabolic\ litter}{FreshN + NO3} \quad C:N = 0\ if\ FreshN + NO3 \leq 0$$
$$C:N = \frac{structural\ litter + metabolic\ litter}{FreshOrganicP + Psol} \quad if FreshOrganicP + LabileP \leq 0$$

[SC.NMN.2]

Where:

- res is the above ground soil residue (kg ha^{-1})
- C is the sum of structural and metabolic litter in the soil layer (kg ha^{-1})
- freshN is the N in the fresh organic pool in the layer (kg ha^{-1})
- NO_3 is the amount of nitrate in the layer
- FreshOrganicP is the P in the fresh organic pool in the layer (kg P ha^{-1})
- LabileP is the amount of P in the solution in the layer (kg P ha^{-1})

The decay rate constant, the same as that used to calculate residue in crop defines the fraction of residue that is decomposed as:

$$Decay = MinCoeff * ResComp * (TempFac * WaterFac)^{0.5}$$
$$ResComp = 1.0$$

[SC.NMN.3]

Where:

- ResComp is the nutrient cycling residue composition factor
- MinCoeff is the rate coefficient for mineralization of the residue fresh organic nutrients (user-defined, default - 0.05)
- temp_factor is temperature's effect on nutrient cycling for the layer
- water_factor is water's effect on nutrient cycling for the layer

Mineralization of the residue fresh organic N (FreshMin) (kg N ha^{-1}) is calculated as:

$$FreshMin = Decay * FreshN$$
$$ResComp = 1.0$$

[SC.NMN.4]

Where:

- Decay is the residue decay rate constant, FreshN is the N in the fresh organic pool in the layer (kg N ha^{-1}). 20% of the N decomposed from the FreshMin is then transferred to the active organic N pool, and 80% is transferred to the NO_3 pool. N in the fresh organic pool is set to 0 in all layers except the top layer. In the top layer, 0.15% of residue is decomposed into fresh organic N pools.

Mineralization (Implemented in humus_mineralization.py)

In general, the term humus describes the transformed products formed from a wide variety of organic substrates without any intent to assign chemical composition or structure (Hayes and Swift, 2020). Humus mineralization allows N to move between the active and stable organic pools. N mineralized from the humus active organic pool is added to the NO_3 pool in the layer.

The amount of N transferred between the active and stable organic pools (Ntrans) (kg N h^{-1}) maintains an equilibrium, and is calculated as

$$amount\_transferred = 0.00001 * active\_organic\_nitrogen * [\frac{1}{FracN} - 1] - stable\_organic\_nitrogen$$

[SC.NMN.5]

Where:

- 10^{-5} is the humus mineralization rate constant
- active_organic_nitrogen is the amount of N in the active organic pool (kg N ha^{-1})
- FracN is the fraction_of_humic_nitrogen_in_the_active_ pool (0.02)
- stable_organic_nitrogen is the amount of N in the stable organic pool (kg N ha^{-1})

When Ntrans is positive, N moves from the active to stable organic pool. When Ntrans is negative, N moves from stable to active organic pool.

D. Phosphorus Cycling

Equations based on the SurPhos model theoretical documentation (Vadas et al., 2007). RuFaS identifies eight different pools of P in the soil where two of them are organic and six inorganic pools. Organic pools are divided into stable and water extractable and inorganic pools are divided into stable, water extractable, active, labile, recalcitrant, and available.

Initialization (Implemented in layer_data.py) The P sorption parameter (PSP) is initialized before the simulation begins. The PSP represents how much of any inorganic P added to soil remains labile P upon reaching relative equilibrium. A PSP of 0.5 means 50% of added P remains labile P and 50% becomes active P.

PSP is initialized as:

$$PSP = max(0.05,\ min(0.7,\ -0.045 * log(clay) + 0.001 * labileP - 0.035 * orgC + 0.43)$$

[SC.PHO.1]

$$adjusted\_clay\_content = max(10^{-8},\ clay\ fraction * 100)$$

[SC.PHO.2]

Where:

- PSP is the P sorption parameter (unitless)
- clay is the adjusted_clay_content
- clay fraction is the fraction of the layer composed of clay
- orgC is the percentage of the layer composed of organic carbon, assumed to be 58% of user-defined organic matter content (%)
- labileP (labile_inorganic_phosphorus) is the concentration of labile inorganic phosphorus (mg kg^{-1})

$$initial\ active\ inorganic\ Pconc = \frac{labile\ inorganic\ P*(1.0-PSP)}{PSP}$$
$$initial\ stable\ inorganic\ Pconc = 4.0 * activeP$$

[SC.PHO.3]

Before the soil P pools are manipulated, they are converted into a kg ha^{-1} basis:

Soluble Phosphorus (Implemented in soluble_phosphorus.py) Calculates the P transferred via hydrological processes in the soil based on equations from the APLE and SurPhos models (Vadas et al., 2007). Soluble P can be lost from the soil as runoff in the first layer and leached vertically down through the soil profile.

Soluble Phosphorus Runoff

$$adjusted\ DRP_{runoff} = min(labileP,\ soilP * extraction\ coefficient * runoff * 10^{-6})$$

[SC.PSL.1]

Where:

- adjusted DRPrunoff is the dissolved reactive phosphorus in runoff from the soil (kg ha^{-1}) surface expressed as the lesser of labileP or calculated dissolved P
- extraction coefficient is 0.005
- Runoff is the amount of water lost from the surface layer (L ha^{-1}) soilP is:

$$soilP = \frac{labile\_phosphorus*10}{bulk\ density*thickness_{cm}}$$

[SC.PSL.2]

Soluble Phosphorus Leaching

Phosphorus leaching is predicted by the clay isotherm function in the APLE documentation (Vadas et al., 2007).

$$isotherm\ slope = 173.51 * clay\ fraction + 8.48$$

[SC.PSL.3]

Where:

- soilP is the concentration of P in the soil layer (mg P kg^{-1} soil)
- Isotherm_slope is the slope of the isotherm curve (unitless)
- Clay fraction is the fractional clay content of a soil layer (unitless)

$$isotherm\ inter = 4.726 * isotherm\ slope - 8.97$$

[SC.PSL.4]

Where:

- Isotherm inter is the intercept of P sorption isotherm (unitless)
- Isotherm slope is the slope of the P sorption isotherm (unitless)

$$DRP_{leachate} = min(20.0,\ exp(\frac{soilP * 1.5 - isotherm\ inter}{isotherm\ slope}))$$

[SC.PSL.5]

Where:

- $DRP_{leachate}$ is the dissolved reactive phosphorus leached from the soil layer (mg L^{-1})

The maximum leachate amount specified in APLE is 20 (mg L^{-1}), specified in the paragraph beneath equation [16] and is applied here as the min term. Leached P is added to the soil layer immediately below the layer for which it is calculated.

The predicted value in (mg L^{-1}) is then converted to kg h^{-1} and the actual dissolved reactive phosphorus is expressed as the minimum of the layer labile P or the calculated $DRP_{leachate}$ to prevent negative values.

$$DRP_{leachate,\ actual} = min(labileP,\ DRP_{leachate\ in\ kg\ ha^{-1}})$$

[SC.PSL.6]

Where:

- $DRP_{leachate}$ is the dissolved reactive leachate P (kg ha^{-1})
- $DRP_{leachate,actual}$ is the actual dissolved reactive P leached (kg ha^{-1})

Summarized leachate values are the sum of values that leave the bottom layer of the soil profile.

Fertilizer Leaching and Runoff (Implemented in fertilizer.py) Simulates the leaching and runoff of phosphorus from fertilizer applied to the soil surface.

Available P pool

During rainfall events, the fraction of fertilizer P lost is greatest during the first rain event and reduced for each consecutive rainfall event. When synthetic fertilizer P is applied, 75% of it goes into an available pool, and 25% goes into a recalcitrant pool. The equation below determines the fraction of P remaining in the available P pool (unitless).

$$fertPavail = max\ (0,\ -0.16 * log(fert_{CNT}) + cov)$$

[SC.PSL.7]

Where:

- $fertP_{avail}$ is the fraction of P remaining in the available P pool
- fertCNT is the number of days since the fertilizer was applied
- cov is the cover factor, adjusted based on cover type:
  - "BARE": 0.5333
  - "RESIDUE COVER": 0.6667
  - "GRASSED": 0.8

Note: the multiplicative and additive constants differ between the SurPhos documentation and here, the ones listed here should be preferred (Vadas et al., 2007).

$fert_{CNT}$ is then iterated to account for the logarithmic curve of fertilizer sorption without rain. The availability of P for runoff or leaching decreases as it sits on the soil surface.

Runoff and Leaching

At the first rainfall event, available P is split between runoff and leachate. Subsequent rain events target the recalcitrant pool. The partitioning of P between runoff and leachate is a function of the amount of rainfall.

During the first rainfall event after fertilizer application, all available fertilizer P is lost to runoff or leached to the next soil layers. For the second event, 40% of released fertilizer P is leached, with all subsequent events leaching 7.5% of available fertilizer P.

$$sorp\ \% = rainday_{fact}$$

[SC.PSL.8]

Where:

- raindayfact is the rain day factor calibrated as described above

Sorbed fertilizer P is then computed and cannot exceed that in the available P pool:

$$fert_{sorp} = min(fertP_{avail},\ max(0.0,\ fertP_{avail} * sorp\ \%))$$

[SC.PSL.9]

$$fertP_{avail} = fertP_{avail} - fert_{sorp}$$

[SC.PSL.10]

$$sorp\% = max\ (0,\ -0.16 * log(fert_{CNT}) + cov)$$

[SC.PSL.11]

Where:

- $Fert_{sorp}$ is the fertilizer sorption

For the first layer, labileP is converted to kg to add adsorbed fertilizer P. labileP is then converted back to kg ha^{-1}. The concentration of fertilizer dissolved P in runoff (mg L^{-1}) ($fertP_{dissolved\ in\ runoff}$) is computed:

$$fertP_{dissolved\ in\ runoff} = \frac{(fertP * fraction\ P\ released * PD_{fact})}{total\ rainfall}$$

[SC.PSL.12]

$$PDfact = 0.034 * exp(3.4 * (\frac{runoff}{rainfall}))$$

[SC.PSL.13]

Where:

- fertP is the amount of fertilizer P in the pool that is going to be leached from (mg)
- fraction of P released is the fraction of P solubilized during the current rain event (unitless).
- PD_{fact} is the P distribution factor is the value that determines P distribution between runoff and infiltration (unitless)
- total rainfall is rainfall on the day (L)

(Vadas et al., 2007) Leached fertilizer P is added to soil labileP for each layer as a function of depth:

$$labileP = labileP + (fert_{leach} - fert_{runoff}) * DF$$

[SC.PSL.14]

Where:

- decreasing fraction of fert P to be leached from one layer to the next

Fertilizer P lost to runoff and leaching are added to their cumulative sums
Manure Phosphorus

Manure is an important source of bioavailable P for crop uptake, runoff, and leaching losses. Organic and inorganic P contributions from manure applications are tracked and modeled, including decomposition of manure solids, labile P release and P sorption. Largely changes to manure P are by three main processes: leaching, decomposition, and assimilation.
Manure Phosphorus Leaching (implemented in manure_pool.py)

$$MIP_{leach} = max(0.0, manure_{extr} * WIP)$$
$$MIP_{leach} = max(0.0, \frac{manure_{extr} * WOP}{0.6})$$

[SC.PSL.31]

Where:

- MIP_{leach} is the manure inorganic P leached (kg ha^{-1}).
- MOP_{leach} is the manure organic P leached (kg ha^{-1})
- WIP is the water extractable inorganic P
- WOP is the water extractable organic P

Leached P is then removed from surface pools and added to respective annual sums.

The concentration of dissolved P in runoff (mg/L):

$$PDfact = (\frac{runoff}{rainfall})^{0.225}$$

[SC.PSL.32]

Where:

- PD_{fact} is the P distribution factor
- runoff is the amount of runoff from rainfall on the current day (mm)
- rainfall is the amount of rainfall on the current day (mm)

The concentration of water extractable phosphorus in runoff on the current day is calculated as

$$MIP_{runoff} = MIP_{leach}\ in\ mg * (\frac{1}{rainfall}) * (\frac{1}{field\ size\ in\ mm^2}) * (\frac{1}{mm^3\ to\ L}) * PD_{fact}$$

[SC.PSL.33]

$$MIP_{leach\ in\ mg} = manure\ leached * 1000000$$

[SC.PSL.34]

$$field\ size_{in\ sq\ mm} = field\ size\ * 10000000000$$

[SC.PSL.35]

Where:

- MIP_{runoff} is the the concentration of water extractable P in runoff on the current day (mg L^{-1})
- MIP_{leach} in mg is the manure leached in mg
- PD_{fact} is the Factor accounting for runoff to rainfall ratio on the current day (unitless)

$$MIO = MOP_{leach\ in\ mg} * (\frac{1}{rainfall}) * (\frac{1}{field\ size\ in\ mm^2}) * (\frac{1}{mm^3\ to\ L}) * PDfact$$

[SC.PSL.36]

Manure runoff P is then converted to kg and the total manure leached P can be calculated with these terms as:

$$M_{leach} = MIP_{leach} - MIP_{runoff} + MOP_{leach} - MOP_{runoff}$$

[SC.PSL.37]

Leached manure P is then added to labileP for each layer as a function of depth (implemented in manure_application.py):

$$labileP = labileP + M_{leach} * DF$$
$$labileP = labileP * DF\ (used\ when\ manure\ is\ applied\ in\ the\ field)$$
$$DF = \frac{ratio}{ratio + exp(-0.867 - (2.078 * ratio))}$$
$$ratio = \frac{center\ depth}{damping\ depth}$$

[SC.PSL.38]

Where:

- DF is the Depth Factor for a given layer of soil, a terminating geometric sequence modeling the decreasing fraction of manure P to be leached from one layer to the next
- center depth is the depth of the center of a given soil layer (mm)
- damping depth is the damping depth of the soil factor (mm)

Manure and leachate P are then added to their respective annual sums

Manure Decomposition (implemented in manure_pool.py)

$$wet_{rate} = -0.3moisture + 0.27$$
$$dry_{rate} = (-0.5 * (\frac{manure_{mass}}{mass}) + 0.075) * T_{fac}$$

[SC.PSL.39]

Where:

- wet_{rate} is the wet rate of manure decomposition (unitless)
- $dry_{rate}$is the dry rate of manure decomposition (unitless)
- $manure_{mass}$is the total mass of manure on the field (kg/ha)
- mass is the mass of applied manure (kg)
- moisture is the manure water content (unitless)

The moisture content of the manure is then adjusted as follows: If rainfall on a given day is above 4 mm:

$$moisture = moisture + wet_{rate},\ if\ rainfall\ \ > \ 4mm$$

[SC.PSL.40]

If rainfall is below 1.0 mm:

$$moisture = moisture - dry_{rate}$$

[SC.PSL.41]

$$moisture = min(0.9,\ max(0.0,\ moisture))$$

[SC.PSL.42]

$$AWDCR = 0.003 * TFA^{0.5}$$
$$ASIM = 30.0 * exp(2.5 * moisture)$$

[SC.PSL.43]

$$TFA = \frac{(2*32^2*T^2 - T^4)}{32^4}$$

[SC.PSL.44]

Where:

- AWDCR is the unitless decomposition factor
- ASIM is the unitless manure assimilation factor. unitless manure factor
- TFA is the unitless temperature factor
- T is the average daily air temperature in $°C$

The decomposition factor, dcom is calculated and then adjusted by $manure_{mass}$

$$dcom = min(max(0.0,\ manure_{mass} * AWDCR),\ manure_{mass})$$

[SC.PSL.45]

The decomposition factor is then used to calculate the cover decomposition factor:

$$cov_{dcom} = min(manure_{cov},\ max(0.0,\ \frac{dcom}{manure_{mass}}))$$

[SC.PSL.46]

The P decomposition factor for each P pool (P_{dcom}) is then calculated:

$$P_{dcom} = min(P,\ max(0.0,\ Ppools * 0.01 * min(TFA,\ moisture)))$$

[SC.PSL.47]

Where:

- P_{pools} represents each of the tracked P pools (SIP, SOP, WOP)

Manure mass and total cover also have assimilation factors calculated as:

$$manure_{ASIM} = min(manure_{mass},\ max(0.0,\ ASIM * TFA * manure_{cov}))$$

[SC.PSL.48]

$$cov_{ASIM} = min(manure_{cov},\ max(0.0,\ \frac{manure_{ASIM}}{manure_{mass} * manure_{cov}}))$$

[SC.PSL.49]

Each pool then also has an ASIM calculated as:

$$P_{ASIM} = min(P,\ max(0.0,\ \frac{manure_{ASIM}}{manure_{mass} * P}))$$

[SC.PSL.50]

Each ASIM and decomposition factor are then removed from the corresponding P pool.

P from decomposition is accounted for in the inorganic pool:

$$WIP = WIP + WOP_{dcom} + SOP_{dcom} * 0.75 + SIP_{dcom}$$
$$WOP = WOP + SOP_{dcom} * 0.25$$

[SC.PSL.51]

The total decomposed P is then:

$$DP = SIP_{ASIM} + WOP_{ASIM} + SOP_{ASIM} + WIP_{ASIM}$$

[SC.PSL.52]

Manure mass and cover are updated to reflect decomposition and assimilation:

$$manure_{mass} = max(0.0,\ manure_{mass} - dcom - man_{ASIM})$$
$$manure_{cov} = max(0.0,\ manure_{cov}\ - cov_{dcom} - cov_{ASIM})$$

[SC.PSL.53]

Decomposed P is then added to the labile pool for each layer as a function of depth:

$$labileP = labileP + DP * DF$$

[SC.PSL.54]

Where:

- DF is the Depth Factor, a terminating geometric sequence modeling the decreasing fraction of decomposed P to be leached from one layer to the next

Runoff (implemented in manure_pool.py)

Manure and soil P in the first layer are susceptible to runoff:

$$soil P is\ the\ labileP \div bulk\ density \div thickness_{cm} \div 0.1$$

[SC.PSL.55]

$$MDRP_{runoff} = soilP * 0.005$$

[SC.PSL.56]

Where:

- $MDRP_{runoff}$ is the manure dissolved reactive P transported in runoff (mg/L)

$$soilP is\ the\ labile P \div bulk\ density \div thickness_{cm} \div 0.1$$

[SC.PSL.55]

$$TIP_{runoff} = MIP_{runoff} + MDRP_{runoff} + fertP_{runoff}$$

[SC.PSL.57]

Where:

- TIP_{runoff} is the total Inorganic P runoff

Phosphorus Mineralization (implemented in phosphorus_mineralization.py)

Phosphorus mineralization is the transfer of P from organic to inorganic pools and between labile and active inorganic pools.

The amount of P that mineralizes into water-extractable inorganic P on the current day from the given pool.

$$mineralization\ rate = rate * min(temp_{factor},\ moisture_{factor})$$

[SC.PMN.1]

Where:

- rate is the rate factor for the type of P being mineralized (unitless).
- $temp_{factor}$ is the temperature factor on the current day (unitless).
- $moisture_{factor}$ is the moisture factor of the given manure pool on the current day (unitless).
- P_{amount} is the amount of P that is mineralized from that pool that is passed (kg)

The maximum P sorption parameter (PSP) is then calculated for each layer then adjusted to be between 0.05 and 0.7 and limited based on the average value of PSP.

$$PSP_{act} = max(0.05,\ min(0.7, (\frac{(meanPSP*364)+currentPSP)}{365}))$$

[SC.PMN.2]

Where:

- PSP_{act} is the actual P sorption parameter (unitless)
- meanPSP is the mean sorption parameter (unitless)
- currentPSP is the current sorption parameter (unitless)

The following variables are used to define the sorption and desorption curves and are empirical.

The current balance of P between labile and active inorganic pools is calculated as pbal. A negative value returned indicates that there is more active phosphorus than there should be, a positive value indicates that there is more labile phosphorus than there should be, and a return value of zero indicates that the two pools are balanced

$$pbal = labileP - activeP * \frac{PSP_{act}}{(1.0 - PSPact)}$$

[SC.PMN.3]

Where:

- pbal is the current balance of P between labile and active inorganic pools
- labileP is the labile inorganic P content of the soil layer (kg/ha)
- activeP is the active inorganic P content of the soil layer (kg/ha)
- PSP_{act} is the The actual P sorption parameter of the layer adjusted from the maximum (unitless)

The P desorption factor (PD_{fac}) calculates how much P should be desorped in the given soil layer and given as:

$$PD_{fac} = base * (day_{count}^{-0.32})$$

[SC.PMN.4]

$$base = (-1.0 * PSP_{act}) + 0.8$$

[SC.PMN.5]

The amount of P that should be removed from the active inorganic P pool to put it in equilibrium with the labile inorganic P pool (kg ha^{-1}) is given as $labile_{pflow}$

$$labile_{pflow} = PD_{fac} * pbal * -1.0$$

[SC.PMN.6]

Where:

- PD_{fac} is the P desorption factor
- day_{count} is the number of days that the active inorganic P pool has been greater than it would be when in equilibrium with the labile inorganic P pool.
- $labile_{pflow}$ is the amount of P moved from the active to labile inorganic P pool to maintain equilibrium (kg/ha).
- base is a value used to determine how much P is transferred from the active P pool to the labile inorganic P pool (unitless).
- PSP_{act} is the actual P sorption parameter of the layer adjusted from the maximum (unitless)

P sorption factor (PD_{fac}) calculates how much P should be sorped in the given soil layer:

$$PS_{fac} = scalar * (day_{count}^{exponent})$$
$$scalar = 0.918 * exp(-4.603 * PSP_{act})$$
$$exponent = -0.238 * log(scalar) - 1.126$$

[SC.PMN.7]

Where:

- PS_{fac} is the P sorption factor
- scalar is the scalar used to calculate sorption factor. The scalar is used in determining how much phosphorus is removed from the labile inorganic phosphorus pool and transferred to the active inorganic phosphorus pool (unitless).
- exponent is a value used as an exponential term when determining the phosphorus sorption rate (unitless).

The amount of phosphorus that should be removed from the labile inorganic phosphorus pool to put it in equilibrium with the active inorganic phosphorus pool (kg / ha) ($active_{pflow}$) is given as:

$$active_{pflow} = PS_{fac} * pbal$$

[SC.PMN.8]

P also transfers from stable pool to the active pool and given as $stable_{pflow}$:

$$stable_{pflow} = 0.0006 * (stableP - (4.0 * activeP))$$
$$stable_{pflow} = min(stablePflow, amounttotransfer) \quad if\ amounttotransfer > 0$$
$$stable_{pflow} = (-1 * amounttotransfer) \quad if\ amounttotransfer > activeP$$

[SC.PMN.9]

Where:

- $Stable_{pflow}$ is the P transferred from the stable to the active inorganic P pool (kg ha^{-1})
- stableP is the stable inorganic P content of the soil layer (kg ha^{-1})
- activeP is the active inorganic P content of the soil layer (kg ha^{-1})

Sediment P Transport in Runoff (implemented in soil_erosion.py)

The following calculation of sediment P transport in runoff is based on SWAT (Neitsch et al., 2011). However, there are differences in how these equations are applied in RuFaS compared to SWAT, and it is recommended that the equations as applied in Rufas be calibrated using observational data.

Organic and inorganic P attached to soil particles are lost to runoff during erosion. The amount of P lost with sediment is calculated with a loading function (McElroy, Wofsy, and Yung, 1977) that was modified by (Williams and Hann, 1978):

E. Carbon Cycling

Decomposition Factors (implemented in decomp_factors.py)

Carbon decomposition is affected by temperature and moisture. The following factors are parameterized and calculated in accordance with the defac spreadsheet assuming coarse soil:

$$T_d = \frac{teff_2 + (\frac{teff_3}{\pi}) * tan^{-1}(\pi * teff_4 * (T_{avg} - teff_1))}{normalizer}$$

[SC.CAR.1]

Where:

- T_d = unitless temperature effect on decomposition factor (empirically determined based on average soil temperature)
- $teff_1$ = x-value at inflection point (empirical factor set at 15.4)
- $teff_2$ = y location of inflection point (before normalization) (empirical factor set to 11.75)
- $teff_3$ = distance from maximum point to the minimum point (empirical factor set at 29.7)
- $teff_4$ = slope of line at inflection point (empirical factor set at 0.031)
- normalizer = empirical factor set at 20.80546

$$M_d = (\frac{water_{fac} - b}{a - b})^{e_1} * (\frac{water_{fac} - c}{a - c}^{e_2})$$

[SC.CAR.2]

Where:

- M_d = unitless moisture effect on decomposition factor (empirically determined based on soil water levels)

- $water_{fac}$ = relative water saturation (%)
- a = dimensionless empirical factor (0.55)
- b = dimensionless empirical factor (1.7)
- c = dimensionless empirical factor (-0.007)
- e_1 = dimensionless empirical factor (6.648115)
- e_2 = dimensionless empirical factor (3.22)

Residue Partitioning (implemented in residue_partitioning.py)

Residue is partitioned into above ground and below ground pools representing stalks and roots left after harvest respectively. Additionally, some above ground residue is incorporated into the soil profile in the case of tillage.

Above Ground: Lignin, Metabolic C, Structural C

Above ground residue is further partitioned into metabolic and structural components based on the lignin composition of the residue.

$$AG_{lignin\ res\ \%} = 0.12 * rainfall * 0.1$$

[SC.CAR.3]

Where:

- $AG_{lignin\ res\ \%}$ = lignin composition of above ground plant residue
- rainfall = amount of rainfall on the current day (mm)

$$AG_{L:N} = \frac{\frac{AG_{lignin\ res\ \%}}{100}}{fr_N} \quad \text{if } fr_N \neq 0$$

Otherwise:

$$AG_{L:N} = 0.4$$

[SC.CAR.4]

Where:

- $AG_{L:N}$ = above ground plant lignin to nitrogen (N) ratio when nitrogen in plant residue at harvest is greater than 0 (unitless)
- fr_N = fraction of N in plant residue at harvest (unitless)

The ratio of lignin to N in residue determines the percentage of above ground residue in the metabolic pool, with a maximum of 85%.

$$AG_{met\ \%} = 0.85 - 0.18 * AG_{L:N}$$

[SC.CAR.5]

Where:

- $AG_{met\ \%}$ = above ground residue that is metabolic (%)

The component of residue dry matter at harvest consisting of above ground metabolic carbon (C) is added to the above ground metabolic C pool.

$$AG_{met} += AG_{DM\ res} * AG_{met\ \%}$$

[SC.CAR.6]

Where:

- AG_{met} = above ground metabolic carbon (kg ha^{-1})
- $AG_{met\ \%}$ = above ground residue that is metabolic (%)
- $AG_{DM\ res}$ = dry matter residue at harvest (kg ha^{-1})

Above ground metabolic C is decomposed into active C and incorporated into the below ground pool via tillage.

$$AG_{met} : C_{active} = AG_{met_{active\ decomp}} * M_d * T_d * AG_{met}$$

[SC.CAR.7]

Where:

- AG_{met}:C_{active} =above ground metabolic C decomposed to active carbon (kg ha^{-1})
- $AG_{met\ active\ decomp}$ = rate of decomposition from metabolic to active C (set at 0.28, Parton et al., 1987)

$$AG_{met} : BG_{met} = AG_{met} * tillage_{\%}$$

[SC.CAR.8]

Where:

- AG_{met}:BG_{met} = above ground metabolic C decomposed to below ground metabolic C (kg ha^{-1})
- tillage = metabolic C incorporated into soil during tillage (unitless)

The above ground metabolic pool is reduced by the amounts decomposed and incorporated.

$$AG_{met} -= AG_{met} : C_{active} + AG_{met} : BG_{met}$$

[SC.CAR.9]

Any dry matter residue at harvest that is not added to the above ground metabolic C pool is added to the above ground structural C pool:

$$AG_{struct} += AG_{DM\ res} * (1 - AG_{met_{\%}})$$

[SC.CAR.10]

Where:

- AG_{struct} = above ground structural C (kg ha^{-1})

Above ground structural C is decomposed into active and slow C pools as well as incorporated into the underground structural C pool via tillage.

$$AG_{struct\ decomp} = K1 * e^{-3} * 1 - AG_{met\ \%}$$

[SC.CAR.11]

Where:

- $AG_{struct\ decomp}$ = rate at which above ground structural C decomposes into slow or active C
- K1 = structural decomposition factor, set at 0.076 (Parton et al., 1987)

$$AG_{struct} : C_{active} = AG_{struct\ decomp} * M_d * T_d * AG_{struct}$$
$$AG_{struct} : C_{slow} = AG_{struct_{decomp}} * M_d * T_d * AG_{struct}$$

[SC.CAR.12]

Where:

- AG_{struct}:C_{active} = above ground structural C decomposed into active C (kg ha^{-1})
- AG_{struct}:C_{slow} = above ground structural C decomposed into slow C (kg ha^{-1})

$$AG_{struct} : BG_{struct} = AG_{struct} * tillage_{\%}$$

[SC.CAR.13]

Where:

- AG_{struct}:BG_{struct} = transfer of structural carbon during tillage (kg ha^{-1})

$$AG_{struct} -= AG_{struct} : BG_{struct} + AG_{struct} : C_{active} + AG_{struct} : C_{slow}$$

[SC.CAR.14]

Below Ground: Lignin, Metabolic C, Structural C

Similarly, the residue below ground is partitioned into metabolic and structural C pools based on the lignin content of the residue. In the case of tillage, some residue is incorporated from the above ground pool.

$$BG_{DM\ res} = AG_{DM\ res} * tillage_{\%}$$

[SC.CAR.15]

In the case of tillage, below ground residue will be a combination of below ground and incorporated biomass.

$$DM_{lignin\ res\ \%} = \frac{BG_{DM\ res}}{BG_{DM\ res} + BG_{bio}}$$

[SC.CAR.16]

Where:

- $DM_{lignin\ res\ \%}$ = weighted fractional amount of lignin in residue dry matter (%)
- BG_{bio} = below ground biomass (kg ha^{-1})

$$BG_{lignin\ res\ \%} = DM_{lignin\ res\ \%} - 0.15 * rainfall * 0.01$$

[SC.CAR.17]

Where:

- $BG_{lignin\ res}$= below ground residue comprised of lignin (%)

The below ground ratio of lignin to N will then be a weighted composition of the lignin to N ratio in the below ground and the incorporated biomass.

$$BG_{L:N} = AG_{L:N} * DM_{ligning\ res\ \%} + (\frac{\frac{BG_{lignin\ res\ \%}}{100}}{fr_N}) * (1 - DM_{lignin\ res\ \%})$$

[SC.CAR.18]

Where:

- $BG_{L:N}$= below ground lignin to N ratio

The component of below ground residue that is metabolic is determined by the below ground lignin to N ratio, with a maximum composition of 85%.

$$BG_{met\ \%} = 0.85 - 0.18 * BG_{L:N}$$

[SC.CAR.19]

Where:

- $BG_{met\%}$= below ground residue that is metabolic (%)

Incorporated metabolic C and the component of below ground biomass consisting of metabolic C are added to the below ground metabolic C pool.

$$BG_{met} += AG_{met} : BG_{met} + (BG_{bio} * BG_{met\ \%})$$

[SC.CAR.20]

Where:

- BG_{met}=below ground metabolic C (kg ha^{-1})

Below ground metabolic C decomposes into active C.

$$BG_{met} : C_{active} = BG_{met_{active\ decomp}} * M_d * T_d * BG_{met}$$

[SC.CAR.21]

Where:

- BG_{met}:C_{active} = below ground metabolic C decomposed into active C (kg ha^{-1})
- $BG_{met\ active\ decomp}$ = rate at which below ground metabolic C decomposes into active C (set at 0.35, Parton et al., 1987)

The decomposed metabolic C is removed from the below ground metabolic C pool.

$$BG_{met} -= BG_{met} : C_{active}$$

[SC.CAR.22]

Incorporated structural and the component of below ground biomass consisting of structural C are added to the below ground structural C pool.

$$BG_{struct} = AG_{struct} : BG_{struct} + BG_{bio} * (1 - BG_{met\ \%})$$

[SC.CAR.23]

Below ground structural C is decomposed into active and slow C.

$$BG_{struct} : C_{active} = BG_{struct\ decomp} * M_d * T_d * BG_{struct}$$
$$BG_{struct} : C_{slow} = BG_{struct_{decomp}} * M_d * T_d * BG_{struct}$$

[SC.CAR.24]

Where:

- BG_{struct}:C_{active} = below ground structural C decomposed into active C (kg ha^{-1})
- BG_{struct}:C_{slow} = below ground structural C decomposed into slow C (kg ha^{-1})
- $BG_{struct\ decomp}$ = rate at which below ground structural C decomposes into slow or active C (set at 0.094, Parton et al., 1987)
- BG_{struct} = below ground structural C (kg ha^{-1})

The decomposed structural C is then removed from the below ground structural pool.

$$BG_{struct} = BG_{struct} : C_{active} + BG_{struct} : C_{slow}$$

[SC.CAR.25]

Pool and Gas Partitioning (implemented in pool_gas_partitioning.py)

There is an imperfect transfer between the metabolic and structural C pools to the active and slow C pools. The amount of C lost to CO2 during the decomposition process is modeled as:

$$AG_{met} : C_{active\ loss} = AG_{met} : C_{active} * \%CO2_{met:active}$$
$$AG_{met} : C_{active\ act} = AG_{met} : C_{active} * (1 - \%CO2_{met:active})$$

[SC.CAR.26]

Where:

- $\%CO2_{met:active}$ = rate of C dioxide loss during transformation of metabolic to active C (0.55, Parton et al., 1987)
- AG_{met}:$C_{active\ loss}$ = above ground metabolic C being lost as carbon dioxide during decomposition into active C (kg ha^{-1})
- AG_{met}:$C_{active\ act}$ =above ground metabolic C decomposed to active carbon after accounting for C dioxide loss (kg ha^{-1})

$$AG_{struct} : C_{active\ loss} = AG_{struct} : C_{active} * \%CO2_{struct:active}$$
$$AG_{struct} : C_{active\ act} = AG_{struct} : C_{active} * (1 - \%CO2_{struct:active})$$

[SC.CAR.27]

Where:

- $\%CO2_{struct\ active}$ = rate of CO_2 loss during transformation of structural to active C (0.45, (Parton et al., 1987))
- $AG_{struct}:C_{active\ act}$ = above ground structural C decomposed to active C after accounting for CO_2 loss C (kg ha^{-1})
- $AG_{struct}:C_{active\ loss}$ = above ground structural C being lost as CO2 during decomposition into active C (kg ha^{-1})

$$AG_{struct} : C_{slow\ loss} = AG_{struct} : C_{slow} * \%CO2_{struct:slow}$$
$$AG_{struct} : C_{slow\ act} = AG_{struct} : C_{slow} * (1 - \%CO2_{struct:slow})$$

[SC.CAR.28]

Where:

- $AG_{struct}:C_{slow\ loss}$ = ecomposing above ground structural C being lost as CO2 during decomposition into slow C (kg ha^{-1})
- $AG_{struct}:C_{slow\ act}$ = above ground structural C decomposed to slow C after accounting for CO2 loss (kg ha^{-1})
- $\%CO2_{struct:slow}$ = rate of CO2 loss during transformation of structural to slow C (0.3, Partonet al. 1987)

$$BG_{met} : C_{active\ loss} = BG_{met} : C_{active} * \%CO2_{met:active}$$
$$BG_{met} : C_{active\ act} = BG_{met} : C_{active} * (1 - \%CO2_{met:active})$$

[SC.CAR.29]

Where:

- $BG_{met}:C_{active\ loss}$ = below ground metabolic C being lost as CO_2 during decomposition into active C (kg ha^{-1})
- $BG_{met}:C_{active\ act}$ = below ground metabolic C decomposed to active C after accounting for CO_2 loss (kg ha^{-1})

$$BG_{struct} : C_{active\ loss} = BG_{struct} : C_{active} * (\%CO2_{struct:active})$$
$$BG_{struct} : C_{active\ act} = BG_{struct} : C_{active} * (1 - \%CO2_{struct:active})$$

[SC.CAR.30]

Where:

- BG_{struct}:$C_{active\ loss}$ = below ground structural C being lost as CO_2 during decomposition into active C (kg ha^{-1})
- BG_{struct}:$C_{active\ act}$ = below ground structural C decomposed to active C after accounting for CO_2 loss (kg ha^{-1})

$$BG_{struct} : C_{slow\ loss} = BG_{struct} : C_{slow} * (\%CO2_{struct:slow})$$
$$BG_{struct} : C_{slow\ act} = BG_{struct} : C_{slow} * (1 - \%CO2_{struct:slow})$$

[SC.CAR.31]

Where:

- BG_{struct}:$C_{slow\ loss}$ = below ground structural C being lost as CO_2 during decomposition into active C (kg ha^{-1})
- BG_{struct}:$C_{slow\ act}$ = below ground structural C decomposed to active C after accounting for CO_2 loss (kg ha^{-1})

The following equations describe the transformation of C among the active, slow, and passive pools, as well as CO2 losses from these pools.

$$C_{active\ decomp\ rate} = K5 * (1 - 0.75 * silt : clay_{\%})$$

[SC.CAR.32]

Where:

- $C_{active\ decomp\ rate}$ = rate at which active C is decomposed into slow C, passive C and CO_2 (%) (kg $year^{-1}$)
- K5 = maximum rate of C decomposition (0.14, Parton et al., 1987)
- silt:$clay_{\%}$ = ratio of percent silt to clay content in the soil

$$C_{active_{decomp}} = C_{active\ decomp\ rate} * M_d * T_d * C_{active}$$

[SC.CAR.33]

Where:

- $C_{active\ decomp}$ = active C decomposed into slow or passive C and CO_2 (kg ha^{-1})
- C_{active} = active C stored in the soil (kg ha^{-1})

$$C_{slow\ decomp} = K6 * M_d * T_d * C_{slow}$$

[SC.CAR.34]

Where:

- $C_{slow\ decomp}$ = slow C decomposed into active or passive carbon and CO_2 (kg ha^{-1})
- K6 = slow C decomposition factor (set at 0.0038, Parton et al., 1987)
- C_{slow} = slow C stored in the soil (kg ha^{-1})

$$C_{passive\ decomp} = K7 * M_d * T_d * C_{passive}$$

[SC.CAR.35]

Where:

- $C_{passive\ decomp}$ = passive C decomposed into active or passive carbon and CO_2 (kg ha^{-1})
- K7 = passive C decomposition factor (set at 0.0038, Parton et al., 1987)
- $C_{passive}$ = passive C stored in the soil (kg ha^{-1})

$$Es = 0.85 - 0.68 * silt : clay_{\%}$$

[SC.CAR.36]

Where:

- Es = adjusted factor of CO_2 loss from the decomposition of active C (Parton et al., 1987)

$$C_{active:slow} = C_{active_{decomp}} * (1 - Es - 0.004)$$
$$C_{active:loss} = C_{active_{decomp}} * Es$$

[SC.CAR.37]

Where:

- $C_{active:slow}$ = active C decomposed into slow C (kg h^{-1})
- $C_{active\ loss}$ = active C lost as CO_2 during decomposition into slow C (kg ha^{-1})

$$C_{active} : C_{passive} = C_{active_{decomp}} * 0.004$$

[SC.CAR.38]

Where:

- C_{active}:$C_{passive}$ = active C decomposed into passive carbon (kg ha^{-1})

$$C_{slow:active} = C_{slow_{decomp}} * (1 - \%CO2 : C_{slow\ loss} - \%C_{slow:passive})$$
$$C_{slow\ loss} = C_{slow\ decomp} * \%CO2 : C_{slow\ loss}$$
$$C_{slow\ passive} = C_{slow\ decomp} * \%C_{slow\ passive}$$

[SC.CAR.39]

Where:

- $C_{slow:active}$ = slow C decomposed into active C (kg ha^{-1})
- %CO_2:C

$$C_{passive\ loss} = C_{passive\ decomp} * \%CO2 : C_{passive\ loss}$$

[SC.CAR.40]

Where:

- $C_{passive:active}$ = passive C decomposed into active carbon (kg ha^{-1})
- %CO_2:C

$$AG : C_{active} = AG_{met} : C_{active\ act} + AG_{struct} : C_{active\ act}$$
$$BG : C_{active} = BG_{met} : C_{active\ act} + BG_{struct} : C_{active\ act}$$
$$C_{active} = AG : C_{active} + BG : +C_{passive} : C_{active} + C_{slow} : C_{active} - C_{active\ decomp}$$

[SC.CAR.41]

Where:

- AG:C_{active} = above ground C decomposed into the active carbon pool (kg ha^{-1})
- BG:C_{active} = below ground C decomposed into the active carbon pool (kg ha^{-1})

$$C_{slow} = AG_{struct} : C_{slow\ act} + BG_{struct} : C_{slow\ act} + C_{active:slow} - C_{slow\ decomp}$$
$$C_{passive} = C_{slow:passive} + C_{active:passive} - C_{passive\ decomp}$$

[SC.CAR.42]

Soil Carbon Aggregation (implemented in carbon_cycling.py)

$$soil\ volume = depth * 10 * area$$
$$soil\ mass = \frac{BD}{0.001} / soil\ volume$$

[SC.CAR.43]

$$C_{active\ \%} = (\frac{C_{active}}{soil\ mass})$$
$$C_{slow\ \%} = (\frac{C_{slow}}{soil\ mass})$$
$$C_{passive\ \%} = (\frac{C_{passive}}{soil\ mass})$$

[SC.CAR.44]

Where:

- C_{active} % = active C composition of the soil (%)
- C_{slow} % = slow C composition of the soil (%)
- $C_{passive}$ % = passive C composition of the soil (%)

$$C_{\%} = C_{active\%} + C_{slow\%} + C_{passive\%}$$

[SC.CAR.45]

Where:

- C% = carbon composition of the soil (%)

$$C = C_{active} + C_{slow} + C_{passive}$$
$$C_{mg} = C * 1000000$$
$$C_g = C * 1000$$

[SC.CAR.46]

Where:

- C = soil carbon (kg ha^{-1})
- C_{mg} = soil carbon (mg ha^{-1})

– C_g = soil carbon (g ha^{-1})

$$AG_{CO2\ loss} = AG_{met} : C_{active\ loss} + AG_{struct} : C_{active\ loss} + AG_{struct} : C_{slow\ loss}$$
$$BG_{CO2\ loss} = BG_{met} : C_{active\ loss} + BG_{struct} : C_{active\ loss} + BG_{struct} : C_{slow\ loss}$$

[SC.CAR.47]

Where:

– $AG_{CO_2\ loss}$ = above ground C lost as CO_2 (kg ha^{-1})
– $BG_{CO_2\ loss}$ = below ground C lost as CO_2 (kg ha^{-1})

$$C_{CO2\ loss\ decomp} = C_{active\ loss} + C_{slow\ loss} + C_{passive\ loss}$$

[SC.CAR.48]

Where:

– $C_{CO_2\ loss\ decomp}$ =C lost as CO_2 during decomposition (kg ha^{-1})

$$C_{CO2\ loss} = AG_{CO2\ loss} + BG_{CO2\ loss} + C_{CO2\ loss\ decomp}$$

[SC.CAR.49]

Where:

– $C_{CO_2\ loss}$ = total C lost as CO_2 (kg ha^{-1})

V. Crop Management

A. Introduction

Crop growth and development in RuFaS is based on the SWAT model and is driven by biomass accumulation based on the available heat units adsorbed by the plant/plant parts, and available nutrient and water uptake from the soil by the crop roots. Absorbed heat units and leaf area index drive potential biomass accumulation and the availability of soil nutrients and water modulate the potential crop growth via stress functions.

We simulate three organic N pools (fresh, active, and stable) and two inorganic pools (NO_3^- and NH_4^+).

Initial NO_3 levels (mg kg^{-1}) in the soil are set by user input. If user input is null, initial nitrate concentrations are generated as:

Planting and Initialization When the Field class plants a new crop, it calls the create_crop() method of the Crop class to initialize a new crop with the appropriate crop configuration inputs. The crop configurations and the default input values for each of the default crop options are provided in the table below. The key inputs to ensure that the crop that is grown will be harvested, stored by the Feed Module, and fed to animals by the Animal Module as intended are:

- Crop category
- Crop type
- RuFaS feed IDs associated with this feed
- Storage type

All other inputs drive the biological processes and ultimately determine the rate and extent of biomass accumulation in response to weather and nutrient availability.

Daily Biophysical Processes The daily crop processes are called by the Field class and are executed in two methods. First the water cycling is executed by the Field class cycle_water() method in which the crop's water cycle is called last. Then the daily crop updates are called for each crop using the Crop class's perform_daily_crop_update() method.

Harvest At the end of the Field class's manage_field() routine, it checks for both user scheduled and optimal harvest events and initiates the Crop routine's harvest if it is a harvest day.

B. Methodology

Crop Water Cycle Daily Updates The water cycle within a crop is executed by methods in the WaterUptake class and the WaterDynamics and are mostly called by the Crop class method cycle_water_for_crop(). First, water is taken up from the soil into the crop by water_uptake.uptake() method and then water is distributed through the crop water cycle by the water_dynamics.cycle_water() method.

The maximum potential water evaporation and transpiration from the crop, however, are called directly from the Field class's cycle_water() method because these values are needed to calculate the field water balance and soil evaporation.

Crop Evaporation The maximum potential crop canopy evaporation and transpiration are calculated as a function of the total potential evapotranspiration for the field on that day (see the Soil documentation).

After the total potential evapotranspiration (mm H_2O) is calculated in the Field class, evaporation from the crop canopy is estimated as:

$$crop\_evaporation = min(crop\_canopy\_water,\ max\_potential\_evapotranspiration)$$

[SC.CRP.1]

Crop Canopy Water The amount of water in the crop's canopy is calculated by the Crop class's handle_water_in_canopy() method. Each crop has a crop-specific maximum water capacity (mm) when the crop is fully matured and this value informs the current day's maximum water storage capacity (mm) based on the ratio of the current leaf area index (LAI) and the maximum LAI:

$$water\_canopy\_storage\_capacity\ =\ max\_water\_capacity * \frac{leaf\_area\_index}{LAImax}$$

[SC.CRP.2]

The potential excess storage capacity is calculated using the maximum water storage capacity and the actual amount of water in the canopy on that day:

$$canopy\_water\_excess\_capacity \; = \; water\_canopy\_storage\_capacity \; - canopy\_water$$

[SC.CRP.3]

The amount of additional water stored in the canopy is then calculated as:

$$water\_taken\_to\_be\_stored \; = \; min(canopy\_water\_excess\_capacity, \; precipitation)$$

[SC.CRP.4]

And added to the current canopy water:

$$canopy\_water = canopy\_water\_previous + water\_taken\_to\_be\_stored$$

[SC.CRP.5]

Finally, the remaining precipitation is passed by the Field class to the Soil as the precipitation that reaches the soil:

$$precipitation\_reaching\_soil \; = precipitation - water\_taken\_to\_be\_stored$$

[SC.CRP.6]

Crop Potential Transpiration The Field's cycle_water method then subtracts the amount of water evaporated from the crop to calculate the remaining evapotranspiration demand (mm H_2O) and that value is passed to the Crop class's method to calculate the maximum crop transpiration (mm H_2O):

$$max\_transpiration = remaining\_evapotranspiration\_demand * leaf\_area\_inde * 3.0 \; if \; 0 \; \leq \; LAI_{act} \leq 3.0$$
$$max\_transpiration = remaining\_evapotranspiration\_demand \; if \; LAI_{act} > 3.0$$

[SC.CRP.6]

Potential Water Uptake The potential water uptake from the soil surface to any specified depth in the root zone can be calculated as:

$$max\_water\_uptake_{depth} = \frac{max\_transpiration}{[1-exp(-water\_dist\_param)]} * [1 - exp(-water\_dist\_param * \frac{depth}{root\_depth})]$$

[SC.CRP.7]

where max_transpiration is the maximum plant transpiration on a given day (mm H_2O), water_dist_param is the water-use distribution parameter, depth is the depth from the soil surface (mm), and root_depth is the depth of root development in the soil (mm).

The potential maximum water uptake from any soil layer, then, can be calculated by taking the difference in uptake between the top and bottom of the soil layer

$$max\_water\_uptakelayer = max\_water\_uptakebottom\_depth - max\_water\_uptaketop\_depth$$

[SC.CRP.8]

where $max\_water\_uptake_{bottom\_depth}$ is the potential water uptake for the lower boundary of the soil layer (mm H_2O) and $max\_water\_uptake_{top\_depth}$ is the potential water uptake for the upper boundary of the soil layer (mm H_2O).

Potential water uptakes from each layer are then calculated sequentially from the soil surface to the maximum depth by layer to allow for compensation for unmet demand set by the maximum water uptake by the available water in lower layers.

The unmet demands for water are calculated as sum of the unmet demands for that layer and all layers above it:

$$unmet\_demand_{layer\_i} = \sum_{x=1}^{i} max\_water\_uptakex - soil\_water$$

[SC.CRP.9]

The unmet demands are then used to calculate the potential water uptakes for each layer from the maximum water uptake and the unmet demands from each layer for every layer below the surface:

$$potential\_water\_uptakelayer = max\_water\_uptake_{layer} + unmet\_demand_{layer} * uptake\_compensation\_factor$$

[SC.CRP.10]

where potential_water_uptake_layer is the adjusted potential water uptake for layer (mm H_2O), uptake_compensation_factor is the plant uptake compensation factor which is a user-defined input for each crop that ranges between 0.01 and 1.00. When the uptake_compensation_factor approaches 1.00 the model allows more of the water uptake demand to be met by lower layers in the soil; when it approaches 0.00, the model allows less variation from the original depth distribution to take place.

As the water content of the soil decreases, the soil holds remaining water more tightly, resulting in a decrease in the efficiency of plant water uptake. Thus, the potential water uptakes are adjusted one more time if the initial soil water content is below a threshold:

$$if\, soil\_water_{layer} < 0.25 \ * available\_water\_capcity_{ayer}:$$
$$adjusted\_potential\_water\_uptake_{layer} =$$
$$potential\_water\_uptake_{layer} * exp[5 * (\frac{soil\_water_{layer}}{0.25 * available\_water\_capcity_{layer}} - 1)]$$

[SC.CRP.11]

where $available\_water\_capcity_{layer}$ is the point at which plant available water in the soil layer begins to limit efficiency of plant uptake, $soil\_water_{layer}$ is the soil water in a given layer (mm H_2O)

Actual water uptake The actual water uptake for each layer are then calculated as the minimum of the adjusted potential water uptakes and the difference between the available soil water in that layer and the wilting point soil water.

$$actual\_water\_uptake\_layer =$$
$$min[adjusted\_potential\_water\_uptake_{layer}, (soil\_water_{layer} - wilting\_point_{layer})]$$

[SC.CRP.12]

where $actual\_water\_uptake\_layer$ is the actual water for the layer (mm H_2O), $wilting\_point_{layer}$ is the water content of the layer at wilting point (mm H_2O).

The total plant uptake for the entire profile for the day (mm H_2O) is calculated as the sum of these values for each layer in the profile.

$$total\_water\_uptake = \sum_{x=1}^{n} actual\_water\_uptake\_later_i$$

[SC.CRP.13]

Where $actual\_water\_uptake\_later_i$ is the actual water for layer (mm H_2O) and n is the number of layers in the soil profile.

The total plant water uptake is passed to WaterDynamics $cycle\_water()$ method as the actual amount of transpiration on the day (mm H_2O).

$$actual\_transpiration = total\_water\_uptake$$

[SC.CRP.14]

Crop Growth and Nutrient Cycle Daily Updates

The second set of daily process updates is called by the execute_daily_processes() method in the Field class which in turn calls perform_daily_crop_update() in the Crop class. Similar to other daily update functions, this method steps through a number of functions to initiate simulation of the biophysical processes that drive crop growth and development. These processes are:

- Check if the crop is mature or is dormant
- Absorb solar radiation in the form of heat units and determine the max potential growth for that day
- Develop roots
- Uptake nitrogen from the soil
- Uptake phosphorus from the soil
- Calculate and set physical constraints to growth
- Grow the canopy of the crop by increasing the leaf area index
- Increase and allocate total crop biomass

Maturity A crop's advance towards maturity is tracked through the accumulation of heat units which are a theoretical unit developed to estimate the amount of solar radiation absorbed by the plant. Each crop has a crop specific value for the potential heat units (PHU) required to reach maturity. The accumulated fraction of potential heat units (FrPHU) on a given day is calculated as:

$$fr_{PHU} = \frac{HU_{sum}}{PHU}$$

[SC.CRP.15]

where HU_{sum} is the heat units accumulated up to the current day and PHU is the crop-specific total heat units required for maturity. HU_{sum} is calculated from day 1 (planting day) to the day until the plant reaches maturity.

Dormancy Perennial crops like alfalfa enter a period of dormancy in which the plants do not grow as the day length nears the shortest or minimum day length for the year. The beginning and end of dormancy are defined by a day length threshold (hrs) which can be calculated as:

$$daylength\_threshold = daylength\_min + dormancy\_threshold$$

[SC.CRP.16]

where $daylength\_min$ is the minimum day length for the location during the year (hrs), $dormancy\_threshold$ is the dormancy threshold (hrs). Plants enter dormancy when the day length falls below the threshold and exit dormancy when day length exceeds the threshold again in the next year.

The dormancy threshold varies with latitude and is calculated as:

$$\begin{gathered} t_{dorm} = 1.0;\ if\ latitude\ >\ 40^\circ\ N\ or\ S \\ t_{dorm} = \frac{(latitude - 20)}{20};\ if\ 20^\circ\ N\ or\ S \leq\ latitude \leq 40^\circ\ N\ or\ S \\ t_{dorm} = 0.0;\ if\ latitude\ <\ 20^\circ\ N\ or\ S \end{gathered}$$

[SC.CRP.17]

where the latitude is expressed as a positive value regardless of cardinality (degrees).

The total daylight hours (daylength, hrs) for each day is calculated by the Weather class each day and passed to the FieldManager:

$$daylength = \frac{2cos^{-1}[-tan(solar\_declination\_radians)tan(latitude)]}{earth\_angular\_velocity}$$

[SC.CRP.18]

where the solar declination in radians is the earth's latitude at which incoming solar rays are normal to the earth's surface and the angular velocity of the earth's rotation is a constant equal to 0.2618 rad h^{-1} or $15^{\circ}\ h^{-1}$.

$$solar\_declination\_radians = sin^{-1}\{0.4 * sin[\frac{2\pi}{year\ length}(day - 82)]\}$$

[SC.CRP.19]

where year length is the number of days in a given year, and the day is Julian day.

At the beginning of the dormant period for perennials (like alfalfa), 10% of the biomass is converted to residue and the LAI for the species is set to the minimum value allowed for the species (default 0.75). For cool season annuals like winter cereal grain species (rye, triticale, wheat, etc), their biomass is not converted to residue. When the day length first drops below the dormancy threshold, the dormancy routine is run once on the first day of the dormancy period.

If it is a perennial crop, the model checks whether the perennial should be cut:

$$if\ (fr_{PHU} \geq fr_{PHU,harvest,min}) : yield$$

[SC.CRP.20]

Where:

- frPHU is the fraction of potential heat units.
- $fr_{PHU,harvest,min}$ is the minimum fraction of potential heat units acquired to warrant harvest (user input).

The parameters that drive growth (fr_{PHU}, HU_{sum}, $fr_{LAI,max}$) are reset to 0 and crop biomass, nitrogen, and phosphorus are reduced by 10% of their value at the time of harvest. Biomass lost when entering dormancy is added to residue.

Accumulation of Heat Units The available heat units on a given day are calculated by the determine_new_heat_units() method in the HeatUnits class:

$$HU = T_{HU} - T_{min};\ when\ T_{HU} > T_{base,min}$$
$$tHU = O;\ if\ T_{HU} - T_{min} < 0$$

[SC.CRP.21]

where T_{HU} is either the mean daily temperature ($^{\circ}C$) or the temperature used to define the maximum heat units for the day; T_{min} is the base or minimum temperature required for the plant growth ($^{\circ}C$).

By default, T_{HU} is set to the mean daily ambient temperature. If the attribute, use_heat_unit_temperature is set to True, then THU is defined by the equation below:

$$T_{HU} = \frac{T_{HU,max}+T_{HU,min}}{2}$$

[SC.CRP.22]

where $T_{HU,max}$ is the maximum heat unit temperature on a given day ($^{\circ}C$), $T_{HU,min}$ is the minimum heat unit temperature on a given day ($^{\circ}C$)

$$T_{HU,min} = T_{crop,min}$$
$$T_{HU,min} = T_{ambient,min};\ if\ T_{ambient,min} > T_{crop,min}$$

[SC.CRP.23]

$$T_{HU,max} = T_{crop,max}$$
$$T_{HU,max} = T_{ambient,max};\ if\ T_{ambient,max} > T_{crop,max}$$

[SC.CRP.24]

where $T_{crop,max}$and $T_{crop,min}$ are the crop-specific maximum and minimum temperatures required to sustain growth ($^{\circ}C$) and $T_{ambient,max}$and $T_{ambient,min}$ are the maximum and minimum ambient temperatures that day.

The effect of using the alternative method on heat unit accumulation varies depending on the relationship between the air temperature range and the crop's growth temperature range:

1. If both min and max air temperatures are higher than the crop's min and max growth temperatures, accumulation is greater than the main method.
2. If both min and max air temperatures are lower than the crop's min and max temperatures, accumulation is greater than the main method.
3. If the air temperature range is entirely within the crop's temperature range, accumulation equals the middle of the crop temperature window.
4. If the crop's temperature range is entirely within the air temperature range, accumulation equals the middle of the air temperature range.

Once the heat units for that day have been calculated, they are added to the previously accumulated heat units by the $add\_heat\_units()$ method.

Root Development The depth of the roots on a given day is estimated based on the fraction of accumulated heat units fr_{PHU} and the maximum rooting depth. First a root specific fraction is calculated:

$$root\_fraction = 0.40 - 0.20 * fr_{PHU}; if\ 2 \geq fr_{PHU} \geq 0$$
$$root\_fraction = 0; if\ 2 < fr_{PHU} < 0$$

[SC.CRP.25]

where $root\_depth_{max}$ is the maximum depth of root development in the soil (mm).

$$root\_depth = root\_depth_{max}$$

[SC.CRP.26]

If the crop type is perennial, and it is not the establishment/planting year then the rooting depth of the given plant (root_depth) (mm) is defined as the depth of root development in the soil on a given day and calculated as:

$$root\_depth = root\_depth_{max}; if\ fr_{PHU} > 0.40$$
$$root\_depth = 2.5 * fr_{PHU} * root\_depth_{max}; if\ fr_{PHU} \leq 0.40$$

[SC.CRP.27]

where root_depthmax is the maximum depth of root development in the soil (mm).

Nitrogen Uptake

Each crop's nitrogen uptake routines are managed by the uptake() method in the NitrogenUptake class. This function executes all nitrogen incorporation routines. It calculates the amount of nitrogen the plant desires based on its current growth stage and the available nitrogen in the soil. Then, it extracts nitrogen from the accessible soil profile. If there's any unmet nitrogen demand, the plant may attempt to fix atmospheric nitrogen. The nitrogen from both extraction and fixation is then added to the plant's biomass, contributing to its growth.

Potential Nitrogen Uptake A crop's demand for nitrogen (and phosphorus), also known as the potential nitrogen uptake, at different stages of development is driven by the optimal composition or fraction of that nutrient in the plant's biomass at that stage of development, the total biomass, and the amount of that nutrient in the plant on the previous day.

The optimal composition of a crop for both nitrogen and phosphorus is predicted based on a curve between the crop-specific, user-provided compositions at emergence ($fr_{N,1}$) and maturity ($fr_{N,3}$).

Following this logic, the function for determining the optimal fraction of N in the plant biomass on a given day (fr_n) is:

$$fr_n = (fr_{N,1} - fr_{N,3}) * [1 - \frac{fr_{PHU}}{fr_{PHU} + exp(n_1 + n_2 * fr_{PHU})}] + fr_{N,3}$$

[SC.CRP.28]

where $fr_{N,1}$ is the fraction of N in the plant biomass at emergence, $fr_{N,3}$ is the fraction of N in the plant biomass at maturity, fr_{PHU} is the fraction of potential heat units accumulated for the plant on a given day in the growing season (described in **[SC.CRP.15])**, and n_1 and n_2 are known as the shape coefficients.

The shape coefficients are calculated by solving the below equations using the composition of the plant at two known points in crop development: half maturity ($fr_{PHU,50\%}$) and maturity ($fr_{PHU,100\%}$). The second shape coefficient is needed to calculate the first shape coefficient and is calculated as:

$$n_2 = \frac{\ln\left(\frac{fr_{PHU,50\%}}{1-\left(\frac{fr_{N,2}-fr_{N,3}}{fr_{N,1}-fr_{N,3}} - fr_{PHU,50\%}\right)}\right) - \ln\left(\frac{fr_{PHU,100\%}}{1-\left(\frac{fr_{N,3}-fr_{N,3}}{fr_{N,1}-fr_{N,3}} - fr_{PHU,100\%}\right)}\right)}{fr_{PHU,100\%} - fr_{PHU,50\%}}$$

[SC.CRP.29]

The first shape coefficient is then calculated as:

$$n_1 = \ln\left[\frac{fr_{PHU,50\%}}{1-(\frac{fr_{N,2}-fr_{N,3}}{fr_{N,1}-fr_{N,3}})} - fr_{PHU,50\%}\right] + n_2 * fr_{PHU,50\%}$$

[SC.CRP.30]

where n_1 is the first shape coefficient, n_2 is the second shape coefficient, $fr_{N,1}$ is the fraction of N in plant biomass at emergence, $fr_{N,2}$ is the the fraction of N in the plant biomass at 50% maturity, $fr_{N,\ 3}$ is the fraction of N in the plant biomass near maturity, $fr_{N,3}$ the normal fraction of N in plant biomass at maturity, $fr_{PHU,50\%}$ is the fraction of potential heat units accumulated for the plant at 50% maturity and $fr_{PHU,100\%}$ is the fraction of potential heat units accumulated for the plant at 100% maturity.

We follow the SWAT model assumption that the difference between the nutrient fraction near maturity and the nutrient fraction at maturity in the crop is equal to 0.00001 and the near mature nutrient fraction is calculated from the mature nutrient fraction according to this assumption.

Using the optimal fraction of N in the crop biomass calculated in **[SC.CRP.28]** the optimal mass of N stored in the plant biomass on a given day (kg N ha^{-1}) is given as:

$$optimal\_nitrogen\_mass = optimal\_nitrogen\_fractionplant\_biomass$$

[SC.CRP.31]

Where optimal_nitrogen_fraction is the optimal fraction of N in the plant biomass for the current growth stage, and plant_biomass is the total plant biomass on a given day (kg ha^{-1}).

Plant N demand (potential N uptake) on a given day (kg N ha^{-1}) is then calculated as:

$$optimal\_nitrogen\_mass = optimal\_nitrogen\_fractionplant\_biomass$$

[SC.CRP.32]

Where optimal_nitrogen_mass_previous is the actual mass of N stored in the plant material at the end of the previous day (kg N ha^{-1}), and biomass_growth_max is the potential increase in total plant biomass on a given day (kg N ha^{-1}), and is constrained by the condition that nitrogen_uptake ≥ 0

Actual Nitrogen Uptake Once the potential nitrogen uptake has been calculated, the request for nutrients from the crops is distributed through the soil layers, starting from the top down. First the soil layers accessible to the roots based on the depth of the roots on that day is determined by the method find_deepest_accessible_soil_layer() which updates each crop's accessible_soil_layers attribute - an integer that indicates the number of soil layers from the surface down to which the crop roots have access.

The number of accessible soil layers is then used to calculate the bottom depths of each layer and the amount of nitrates in each layer that are accessible to the plant by the methods access_layers in the NonWaterUptake class.

The potential N uptake from the surface to any given depth z ($N_{up,z}$; kg N ha^{-1}) is calculated as:

$$nitrogen\_uptake_{depth} = \frac{nitrogen\_uptake}{[1-exp(-\beta_n)]} * 1 - exp(-\beta_n * \frac{depth}{root\_depth})$$

[SC.CRP.33]

where nitrogen_uptake_depth is the potential N uptake from the soil surface to the lower boundary of a given depth, nitrogen_uptake is the total potential N uptake (kg N ha^{-1}), β_n is the nutrient uptake distribution parameter, "depth" is the depth from soil surface (mm), and root_depth is the depth of root development in the soil on a given day (mm).

The β_n parameter does not significantly affect N uptake by plants, but it has a significant impact on the maximum amount of NO_3 removed from the upper 10 cm via surface runoff. Higher β_n values allow plants to extract a greater percentage of N_3up from the upper soil layers, which results in less NO_3 loss to runoff. We set this parameter to a default value of 10.

Potential N uptake for each layer is then calculated as the difference in the potential uptake to the bottom depth of the layer and the potential uptake to the top depth of each layer. For the surface layer, where the top depth or upper boundary of the soil layer is at 0 mm depth:

$$potential\_nitrogen\_uptake_{layer} = nitrogen\_uptake_{depth}$$

[SC.CRP.34]

where $nitrogen\_uptake_{layer}$ is the potential N uptake from the surface soil layer (kg N ha^{-1})

For every other layer, the potential N uptake is calculated for the upper and lower boundaries of the layer and $N_{up,ly}$ is taken as the difference between the two:

$$potential\_nitrogen\_uptake_{layer} = nitrogen\_uptake_{depth,bottom} - nitrogen\_uptake_{depth,upper}$$

[SC.CRP.35]

where potential_nitrogen_uptake$_{depth,bottom}$ is the potential N uptake from the soil surface to the lower boundary of the soil layer (kg N ha^{-1}) and nitrogen_uptake$_{depth,upper}$ is the potential N uptake from the soil surface to the upper boundary of the soil layer (kg N ha^{-1}).

The actual N uptake from a given soil layer (kg N ha^{-1}) is calculated as the minimum of potential N uptake plus demand, and available NO_3 in the layer and given as:

$$actual\_nitrogen\_uptake_{layer} = \\ min\{nitrogen\_uptake_{layer} + unmet\_nitrogen\_demand,\ soil\_nitrate_{layer}\}$$

[SC.CRP.36]

where unmet_nitrogen_demand is the potential N uptake from the overlying soil layers (kg N ha^{-1}) that was not met by the nutrients in those layers. For the first soil layer where there are no overlying layers, unmet_nitrogen_demand is 0. nitrogen_uptake$_{layer}$ is the potential N uptake for the soil layer (kg N ha^{-1}), and soil_nitrate is the nitrate content of the soil layer (kg N ha^{-1}).

For every other soil layer the unmet nitrogen demand is calculated as:

$$unmet\_nitrogen\_demand = max(sum\_potential\_nitrogen\_uptake_{above} - sum\_soil\_nitrate_{above}, 0)$$

[SC.CRP.37]

where N_{up},over is the sum of potential N uptake for each layer overlying the given layer (kg N ha^{-1}) and NO^3_{over} is the sum of nitrate content of each soil layer overlying the given layer (kg N ha^{-1}).

$$sum\_potential\_nitrogen\_uptake_{above} = \sum_{layer-1}^{surface} potential\_nitrogen\_uptake_{layer}$$
$$sum\_soil\_nitrate_{above} = \sum_{layer-1}^{surface} soil\_nitrate_{layer}$$

[SC.CRP.38]

where layer -1 is the soil layer above the current layer, S is the surface layer, $potential\_nitrogen\_uptake_{layer}$ is the potential uptake for the given soil layer (kg N ha^{-1}),$soil\_nitrate_{layer}$ is the nitrate content of the given soil layer (kg NO_3-N ha^{-1})

$actual\_nitrogen\_uptake_{total}$ for the entire profile on a given day is the sum of N

$$actual\_nitrogen\_uptake_{total} = \sum_{z}^{s} actual\_nitrogen\_uptake_{layer}$$

[SC.CRP.39]

Nitrogen Fixation The N fixation module amends the routine and equations described in the SWAT 2009 documentation by limiting calculations of f_{NO3}, f_{SW}, N_{demand}, and N_{fix} to the portion of the soil profile that is accessible to root biomass. This module also only applies to those crops that have been designated/defined as capable of forming symbiotic N fixation associations (e.g. legumes).

After the amount of N available in the soil to meet the crop's nitrogen demands has been determined, the amount of N added to the plant biomass by fixation (kg N ha^{-1}) is calculated using the unmet demands from the soil:

$$nitrogen\_fixation = unmet\_demand * stage\_factor * min\{water\_factor,\ nitrate\_factor,\ 1\}$$

[SC.CRP.40]

where unmet_demandthe plant N demand not met by uptake from the soil (kg N ha^{-1}) and is also the maximum amount of N that can be fixed by the plant on a given day, stage_factor is the growth stage factor (0.0-1.0), $water\_factor$ is the soil water factor (0.0-1.0), and nitrate_factor) the soil nitrate factor (0.0-1.0).

The growth stage factor is dependent on the fraction of accumulated potential heat units (PHU) for the plant on a given day in the growing season and can be expressed as:

$$\begin{array}{cc} stage\_factor = 0; & if\ fr_{PHU} \leq 0.15 \\ stage\_factor = 6.67 * fr_{PHU} - 1; & if\ 0.15 < fr_{PHU} \leq 0.30 \\ stage\_factor = 1; & if\ 0.30 < fr_{PHU} \leq 0.55 \\ stage\_factor = 3.75 - 5 * *fr_{PHU}; & if\ 0.55 < fr_{PHU} \leq 0.75 \\ stage\_factor = 0; & if\ fr_{PHU} > 0.75 \end{array}$$

[SC.CRP.41]

stage_factor reflects the growth and decline of N fixing bacteria in the plant roots during the growing season.

The soil nitrate factor is dependent on the sum of nitrate available in the soil layers accessible to root biomass (kg NO_3-N ha^{-1}) and calculated as:

$$accessible\_nitrates = \sum_{i=1}^{root\_layer} soil\_nitrates_{layer,i}$$

[SC.CRP.42]

where root_layer is the soil layer of lowest depth that is accessible to the root biomass, and $soil\_nitrates_{layer,i}$ is the nitrate content in the given soil layer (kg N ha^{-1}).

The soil nitrate factor is then determined along the continuum:

$$\begin{array}{cc} nitrate\_factor = 1; & if\ accessible\_nitrates \leq 100 \\ nitrate\_factor = 1.5 - 0.0005 * NO3_{root}; & if\ 100 < accessible\_nitrates \leq 300 \\ nitrate\_factor = 0; & if\ accessible\_nitrates > 300 \end{array}$$

[SC.CRP.43]

The soil water factor controls N fixation as the soil dries out and is calculated in the SoilData class:

$$water\_factor = \frac{root\_accessible\_soil\_water}{0.85 * root\_accessible\_field\_capacity}$$

[SC.CRP.44]

where root_accessible_soil_water is the soil water content in the soil profile accessible to plant root biomass (mm H_2O) and root_accessible_field_capacity is the soil water content accessible to root biomass at field capacity (mm H_2O).

root_accessible_soil_water and $0.85 * root\_accessible\_field\_capacity\_t$ are calculated as the sum of total soil water and soil water field capacity in each layer for the soil layers accessible to the root biomass.

Store Nitrogen in Plant The actual nitrogen stored in plant biomass on a given day (kg N ha^{-1}) is calculated as:

$$crop\_nitrogen = previous\_crop\_nitrogen + N_{actualup} + N_{fix}$$

[SC.CRP.45]

where $previous\_crop\_nitrogen$ is the N stored in plant biomass on the previous day (kg N ha^{-1}), $N_{actualup}$ is the actual N uptake for the entire soil profile (kg N ha^{-1}), andN_{fix} is the amount of N added to the plant biomass by fixation (kg N ha^{-1}) for perennial crops.

Phosphorus Uptake

The methods for estimating crop phosphorus uptake and addition to the plant's stored phosphorus are analogous to the methods for crop nitrogen uptake and both the PhosphorusUptake class and the NitrogenUptake class use the same generic methods that are inherited from the NonWaterUptake class to implement these calculations. The pool of phosphorus available for uptake by the crop is labile inorganic phosphorus.

To summarize:

- First the potential phosphorus uptake is estimated based on the demand from the crop given its current biomass, stage of development, and estimated phosphorus composition at that stage of development. These methods are described in **[SC.CRP.28] - [SC.CRP.32]** substituting phosphorus for nitrogen.
- Then the actual phosphorus uptake is calculated based on the number of soil layers that the roots have access to and the amount of labile inorganic phosphorus in each soil layer. These methods are described in **[SC.CRP.33] - [SC.CRP.44]** again, substituting phosphorus for nitrogen.
- Finally the phosphorus taken up by the roots are added to the amount of phosphorus stored in the plant as described in **[SC.CRP.45]** with the obvious exception that there is never any fixed phosphorus.

Growth Constraints

The GrowthConstraints class includes all of the methods that estimate the potential limitations on plant growth that a crop faces due to environmental conditions. Specifically, these methods calculate a stress factor for each of the following:

- Insufficient or excess water
- Inadequate nitrogen
- Inadequate phosphorus
- Extreme temperature

The stress factors have values between [0,1] and represent the proportional reduction in total potential biomass that is accumulated that day as a result of that environmental stressor. After calculating the stress factor for each condition separately, the largest stressor is used to define the growth factor for that day:

$$growth\_factor = 1 - max(water\_stress,\ temp\_stress,\ nitrogen\_stress,\ phosphorus\_stress)$$

[SC.CRP.46]

If the user wishes to run a simulation in which crop growth ignores these environmental stressors, they can do so through the user inputs "simulate_water_stress", "simulate_temp_stress", "simulate_nitrogen_stress", and "simulate_phosphorus_stress" in each field. The default values for these are set to true, indicating that crop growth should take environmental stressors into account but setting any of these input values to false will "turn off" that stressor and return a value of 0 for the associated stress factor.

Water stress Water stress is the stress caused by the soil water conditions on a given day.
The water stress for a given day is calculated as:

$$water\_stress = 1 - \frac{w_{actualup}}{E_t}$$

[SC.CRP.47]

where $w_{actualup}$ is the daily total plant water uptake (mm H_2O) and E_t is the actual amount of transpiration on a given day (mm H_2O).

Temperature stress Temperature stress is experienced by the crops when the mean air temperature on a given day diverges from the optimal temperature for the plant growth. If the average temperature is less than or equal to the minimum temperature required for crop growth, also called the base temperature, then the temperature stress for that day will be 1 and the crop will not accumulate any biomass.

$$temp\_stress = 1; \qquad if\ \bar{T}_{ave} \leq T_{base}$$

[SC.CRP.48]

where $\bar{T}_{ave}$ is the mean air temperature ($^\circ C$) and T_{base} is the plant-specific minimum temperature for growth ($^\circ C$).
If the base temperature is less than the average temperature and the average temperature is less than or equal to the optimal temperature, then temperature stress is calculated as:

$$temp\_stress = 1 - exp[\frac{-0.1054(T_{opt}-\bar{T}_{ave})^2}{(\bar{T}_{ave}-T_{base})^2}]; \quad if\ T_{base} < \bar{T}_{ave} \leq T_{opt}$$

[SC.CRP.49]

where T_{opt} is the plant-specific optimal temperature for growth ($^\circ C$) If the optimal temperature is less than the average temperature and the average temperature is less than or equal to two times the optimal temperature minus the base temperature, then temperature stress for a given day is calculated as:

$$temp\_stress = 1 - exp[\frac{-0.1054(T_{opt}-\bar{T}_{ave})^2}{(2*T_{opt}-\bar{T}_{ave}-T_{base})^2}]; \quad if\ T_{opt} < \bar{T}_{ave} \leq 2*T_{opt}-T_{base}$$

[SC.CRP.50]

If the average temperature is greater than two times the optimal temperature minus the base temperature, then temperature stress is set to 1 and results in no biomass accumulation,

$$temp\_stress = 1; \qquad if\ \bar{T}_{ave} > 2*T_{opt}-T_{base}$$

[SC.CRP.51]

Nitrogen and Phosphorus Stress Nitrogen and phosphorus stress are calculated using the same functions based on a non-linear relationship between the current nitrogen content of the plant and the optimal content of the plant on that day. Nitrogen stress, however, is not calculated only for legumes. Both nitrogen and phosphorus stress are 0.0 at optimal nutrient content and 1.0 when the nutrient content of the plant is 50% or less of the optimal value.

First a scaling factor is calculate that relates the current plant nutrient content to the optimal nutrient content:

$$scaling\_factor = 200(\frac{stored\_nutrient\_content}{optimal\_nutrient\_content} - 0.5); \qquad if\ optimal\_nutrient\_content \neq 0$$

[SC.CRP.52]

The nutrient stress for a given day is then calculated as:

$$\begin{gathered} nutrient\_stress = \\ 1(\frac{scaling\_factor}{scaling\_factor + exp[3.535 - 0.02597 * scaling\_facotr]'}; \qquad if\ optimal\_nutrient\_content \neq 0 \\ nutrient\_stress = 0; \qquad if\ optimal\_nutrient\_content = 0 \end{gathered}$$

[SC.CRP.53]

where stored_nutrient_content is the actual mass of nitrogen or phosphorus stored in plant material on the current day (kg N or P ha^{-1}) and optimal_nutrient_content is the optimal mass of nitrogen or phosphorus stored in plant material for the current growth stage (kg N or P ha^{-1}).

Grow Canopy

The amount of canopy cover is expressed as the leaf area index (LAI). LAI is defined as the area of green leaf per unit area of land. Each crop species has a maximum LAI and once that maximum is reached, the LAI will remain constant until leaf senescence begins to exceed leaf growth.

LAI is calculated using equations that relate the current fraction of accumulated heat units and crop-specific heat unit fractions associated with different stages of development. The methods for calculating the LAI are found in the LeafAreaIndex class.

First, two unitless shape coefficients, shape_1 and shape_2, are calculated by the method determine_lai_shapes():

$$shape\_1 = \ln[\frac{fr_{PHU,1}}{fr_{LAI,1}} - fr_{PHU,1}] + shape\_2 * fr_{PHU,1}$$

[SC.CRP.54]

$$shape\_2 = \frac{[\ln(\frac{fr_{PHU,1}}{fr_{LAI,1}} - fr_{PHU,1}) - \ln(\frac{fr_{PHU,2}}{fr_{LAI,2}} - fr_{PHU,2})]}{fr\_PHU,2 - fr_{PHU,1}}$$

[SC.CRP.55]

where $fr_{PHU,1}$, $fr_{PHU,2}$, $fr_{LAI,1}$, and $fr_{LAI,2}$ are the crop-specific curve points. Specifically, $fr_{PHU,1}$ and $fr_{PHU,2}$ are the fractions of total potential heat units of the growing season corresponding to the

1st point and 2nd point on the optimal leaf area development curve and $fr_{LAI,1}$ and $fr_{LAI,2}$ are the fractions of the maximum plant LAI corresponding to the 1st and 2nd point on the optimal leaf area development curve.

Next, the optimal LAI fraction for a given day is calculated by the method determine_optimal_leaf_area_fraction() as:

$$fr_{LAI,opt} = \frac{fr_{PHU}}{fr_{PHU} + exp(shape\_1 - shape\_2 * fr_{PHU})}$$

[SC.CRP.56]

where $fr_{LAI,opt}$ is the fraction of the plant's maximum potential LAI corresponding to a given fraction of potential heat units (PHU) the plant has accumulated on that day.

The optimal LAI fraction ($fr_{LAI,opt}$) is then used by the method determine_canopy_height() to set the canopy height for that day according to the following equation:

$$canopy\_height = min(canopy\_height_{max},\ canopy\_height_{max} * \sqrt{fr_{LAI,opt}})$$

[SC.CRP.57]

The optimal increase in LAI for annuals and perennials that are not in senescence is calculated by the method determine_max_leaf_area_change() using the following equation:

$$optimal\_leaf\_area\_index\_change = (fr_{LAI,opt,d} - fr_{LAI,opt,d-1}) * LAI_{max} * (1 - exp(5 * (LAI_{act,d-1} - LAI_{max})))$$

[SC.CRP.58]

where LAImax is the crop-specific maximum LAI, $fr_{LAI,max,d}$ and $fr_{LAI,max,d-1}$are the fraction of the plant's maximum LAI accumulated on the given day d and the previous day respectively, and $LAI_{act,d-1}$LAIact,d-1 is the actual LAI on the previous day.

The actual LAI added is then calculated as:

$$actual\_leaf\_area\_index\_change = optimal\_leaf\_area\_index\_change * \sqrt{growth\_factor}$$

[SC.CRP.59]

where actual_leaf_area_index_change is the change in the LAI the given day and growth_factor is the plant's growth factor (0.0 - 1.0) calculated in **[SC.CRP.46]**.

The total LAI for the plant that day is then updated according to the following:

$$leaf\_area\_index = previous\_leaf\_area\_index + actual\_leaf\_area\_index\_change$$

If the plant is in a stage of senescence, the LAI for annuals is calculated as:

$$leaf\_area\_index = LAI_{max} * (\frac{1-fr_{PHU}}{1-fr_{PHU,sen}})$$

[SC.CRP.60]

where LAI_{max} is the maximum LAI, $fr_{PHU,sen}$ is the crop-specific fraction of PHU at which senescence becomes the dominant growth process, and fr_{PHU} is the fraction of PHU accumulated for the plant on a given day in the growing season.

Biomass Accumulation

Plant biomass is produced when intercepted solar radiation is converted into plant biomass via photosynthesis assuming a plant species-specific radiation use efficiency. The methods that estimate the conversion of solar radiation into plant biomass are collected in the BiomassAllocation class.

The amount of photosynthetically active radiation (MJ m^{-2}) that is intercepted by the leaf area is calculated by the method intercept_radiation using the following function:

$$intercepted\_radiation = 0.5 * radiation * (1 - exp(-extinction\_coef * LAI_{act}))$$

[SC.CRP.61]

where radiation is the incident total solar radiation (MJ m^{-2}), 0.5 X H_{day}, extinction_coef is the light extinction coefficient, and LAI_{act} is the leaf area index.

Using this value, the maximum potential biomass increase for that day is calculated as:

$$biomass\_growth\_max = light\_use\_efficiency * intercepted\_radiation$$

[SC.CRP.62]

where biomass_growth_max is the potential increase in total plant biomass on a given day (kg ha^{-1}), light_use_efficiency is the crop-specific radiation use efficiency (10^{-1} g MJ^{-1} or kg ha^{-1} x MJ m^{-2}), and intercepted_radiation is the amount of intercepted photosynthetically active radiation on a given day (MJ m^{-2}).

The actual increase in biomass (kg/ha) on a given day) is calculated as:

$$biomass\_growth = biomass\_growth\_max * growth\_factor$$

[SC.CRP.63]

Where growth_factor is the plant growth factor (0.0 - 1.0) calculated in **[SC.CRP.46]**.

Biomass Allocation

Once the total biomass growth for a given day has been calculated, the total accumulated biomass can be partitioned into above and below ground biomass depending on the development stage of the crop. The aboveground biomass (kg hha^{-1}) is calculated as:

$$aboveground\_biomass\_growth = (1 - root\_fraction) * biomass\_growth$$

[SC.CRP.64]

where root_fraction is the fraction of total biomass in the roots calculated in **[SC.CRP.25]** and $biomass\_growth$ is the actual plant biomass added that day (kg hha^{-1}).

$$belowground\_biomass\_growth = root\_fraction * biomass\_growth$$

[SC.CRP.65]

Crop Harvest

When the Field class initiates a harvest event via the manage_harvest() function of the Crop class, the CropManagement class method for harvesting is called which:

- Determines the harvest index for that crop which accounts for the type of feed and storage of that crop
- Cuts and collects the crop's harvested biomass according to the harvest index and harvest efficiency
- Kills the crop if it is an annual or the last harvest of a perennial crop
- Records the yield
- Transfers the residue to the soil

Harvest Index

If a custom harvest index is provided by the user (harvest index override), that value is used. Otherwise, the harvest index is calculated based on the crop's accumulated heat fraction and the crop-specific optimal harvest index. The method also adjusts the harvest index based on the crop's water deficiency. Harvest index is defined as the fraction of the above ground dry biomass of plant that is removed as dry yield. The potential harvest index (HI_{max}) for the plant for a given day is calculated as:

$$potential\_harvest\_index = optimal\_harvest\_index * \frac{100 * fr_{PHU}}{[100 * fr_{PHU} + exp(11.1 - 10 * fr_{PHU})]}$$

[SC.CRP.66]

where $optimal\_harvest\_index$ is the optimal harvest index for the plant at maturity, given ideal growing conditions and fr_{PHU} is the fraction of potential heat units accumulated for the plant on a given day in the growing season.

The actual harvest index is then adjusted for water deficiency:

$$actual\_harvest\_index = (potential\_harvest\_index - minimum\_harvest\_index) * \frac{water\_deficiency}{water\_deficiency + exp(6.13 - 0.883 * water\_deficiency)} + HI_{min}$$

[SC.CRP.67]

where potential_harvest_index is the potential harvest index, minimum_harvest_index is the harvest index for the plant in drought conditions and represents the minimum harvest index allowed for the plant, water_deficiency is the water deficiency factor.

As the water deficiency factor increases, the actual harvest index will decrease. The water deficiency factor is calculated in the WaterDynamics class during the cycle_water() method:

$$water\_deficiency = * \frac{\sum_{p}^{m} actual\_evapotranspiration}{\sum_{p}^{m} potential\_evapotranspiration}$$

[SC.CRP.68]

where p is the day of planting, m is the day of harvest if the plant is harvested before it reaches maturity or the last day of the growing season if the plant is harvested after it reaches maturity, actual_evaoptranspiration is the actual evapotranspiration on a given day (mm H_2O), and potential_evaoptranspiration is the potential evapotranspiration on a given day (mm H_2O).

Crop Cutting

After the harvest index has been set, the manage_harvest method calls the cup_crop() method that determines how much biomass is cut from the growing crop. The proportion of a crop that is cut is determined by the harvest index. A harvest index $<$ 1 indicates that a proportion of aboveground biomass equal to the harvest index will be removed from the aboveground biomass of the plant. A harvest index $>$ 1 will remove below ground biomass as well.

The cut biomass is removed from the plant's total biomass and the amount collected as yield is determined by the collected fraction. The remaining portion is left in the field as the crop residue created during harvest. Cut operations without a harvest, as in the case of cover cropping systems, are conducted by setting collected_fraction = 0 (the default).

If the harvest index is less than one, the cut biomass (kg ha^{-1}) is calculated as:

$$cut\_biomass = aboveground\_biomass * actual\_harvest\_index$$

[SC.CRP.69]

The actual harvested yield (kg DM ha^{-1}) and cut crop residue (kg DM ha^{-1}) left on the field are then calculated using the harvest efficiency which is defined as the fraction of cut biomass removed by the harvesting equipment.

$$dry\_yield\_collected = cut\_biomass * harvest\_efficiency$$
$$yield\_yield\_collected = cut\_biomass * (1 - harvest\_efficiency)$$
$$biomass_{update} = biomass_{previous} - cut\_biomass$$

[SC.CRP.70]

After the total biomass and yields have been updated, the above and belowground biomass in the plant material that remains in the field are updated along with the plant's root fraction via the recalculate_biomass_distribution() method.

If no roots are harvested:

$$aboveground\_biomass_{update} = aboveground\_biomass_{previous} - cut\_biomass$$
$$root\_fraction = \frac{root\_biomass}{biomass_{update}}$$

If roots are harvested:

$$root\_biomass\_removed = cut\_biomass - aboveground\_biomass_{previous}$$
$$root\_biomass_{update} = root\_biomass_{previous} - root\_biomass\_removed$$
$$aboveground\_biomass_{update} = 0$$
$$root\_fraction = 0$$

[SC.CRP.71]

In addition to total biomass, the nutrient content for both the collected and uncollected portions are updated. If the simulation is using the internally-derived harvest index for cutting, then nutrients are determined with the crop-specific yield nutrient fractions. Otherwise, (harvest index override), the optimal nutrient values are used.

The amount of nitrogen and phosphorus removed with the collected biomass, or the nitrogen and phosphorus yields (kg N or P ha^{-1}) are calculated as:

$$nitrogen\_yield = nitrogen\_fraction * dry\_yield\_collected$$
$$phosphorus\_yield = phosphorus\_fraction * dry\_yield\_collected$$

[SC.CRP.72]

where $nitrogen\_fraction$ is the fraction of nitrogen in the harvested biomass, and $phosphorus\_fraction$ fraction of phosphorus in the harvested biomass.

The cut_crop() method updates the crops leaf area index and accumulated heat units. This is especially important for perennial crops that will continue growing either the following day or after the end of their dormancy.

The plant's leaf area index and accumulated heat units are set back by the fraction of biomass removed in yield.

$$fraction\_cut = \frac{cut\_biomass}{aboveground\_biomass}$$

[SC.CRP.73]

$$leaf\_area\_index_{update} = leaf\_area\_index_{previous} * (1 - fraction\_cut)$$

[SC.CRP.74]

$$accumulated\_heat\_unit_{update} = accumulated\_heat\_unit_{previous} * (1 - fraction\_cut)$$

[SC.CRP.75]

Reducing the number of accumulated heat units shifts the plant's development to an earlier period in which growth is usually occurring at a faster rate.

Crop Kill

If the plant is killed at harvest, the plant biomass that was not removed by harvest and the nutrients in that biomass are added to the residue pools (kg DM, N or P ha^{-1}). So after the cut_crop() methods calculates the yield and the residue produced by the harvest practice, the kill() method, updates the status of the crop plant to indicate that it is no longer living and adds the remaining plant biomass and nutrients to the yield residue pools.

$$\begin{gathered} yield\_residue_{update} = yield\_residue_{previous} + biomass \\ yield\_nitrogen\_residue_{update} = yield\_residue_{update} * nitrogen\_fraction \\ yield\_phosphate\_residue_{update} = yield\_residue_{update} * phosphorus\_fraction \end{gathered}$$

[SC.CRP.76]

Record Harvest and Transfer Residue

The last steps in the CropManagement method for managing the crop harvest is to record the harvested crop information in a data structure that can be passed to the OutputManager and the Feed Storage Module and then to add the crop residue to the soil residue pools. The latter is accomplished by the transfer_residue method.

If the crop is not killed and there is only surface residue and the soil pools in the surface layer are updated as follows:

$$\begin{gathered} soil\_plant\_residue = yield\_residue \\ fresh\_organic\_nitrogen\_content_{update} = \\ fresh\_organic\_nitrogen\_content_{previous} + yield\_nitrogen\_residue \\ labile\_inorganic\_phosphorus\_content_{update} = \\ labile\_inorganic\_phosphorus\_content_{previous} + yield\_phosphorus\_residue \end{gathered}$$

[SC.CRP.77]

If the crop is killed and there is both surface and belowground crop residue to add to the soil pools, the residue is distributed between the soil layers by the distribute_residue_nutrients() method.

First, the fraction of the residue that remains on the surface is calculated as:

$$surface\_residue\_fraction = \frac{yield\_residue - root\_biomass}{yield\_residue}$$

The surface residue, nitrogen residue, and phosphorus residues are then calculated by multiplying the total residue masses by the surface residue fractions and that mass is added to the soil pools as described above in **[SC.CRP.77]**.

Then the residue masses added to the subsurface soil layers calculated as the masses multiplied by (1-surface_residue_fraction). The subsurface residue masses are distributed to different soil layers by calculating the distribution of root biomass between the soil layers following a method described by Fan et al., 2016. The work by Fan et al., 2016 propose a logistic dose response curve for the cumulative distribution of the root biomass up to a specific depth in the soil profile. We adapt the methods they proposed to estimate the proportion of biomass in each soil layer, rather than the cumulative proportion at a given soil depth.

Heuristically, the method is executed as follows:

Starting with the top layer:

1. Estimate the proportion of the root biomass distributed to the depth that is equal to the bottom of the soil layer using Equation 2 in Fan et al., 2016 reproduced in **[SC.CRP.78]** below.
2. Multiply the proportion of the root biomass distributed up to the depth of the current soil layer from step 1 by remaining yield residue masses (total mass, nitrogen mass, and phosphorus mass).
3. Subtract the sum of the yield residue mass for all layers above the current layer from the total yield residue mass from the surface to the bottom of the soil layer to get the root biomass in that latter alone. (note, for the first sub-surface layer, this sum will be zero).

Repeat for each layer going down. The equation from Fan et al., 2016 that determines the fraction of the root biomass up to a given depth is described below and broken up into 3 parts for clarity:

$$root\_biomass\_layer\_fraction = first\_term + (second\_term * third\_term)$$
$$first\_term = \frac{1}{1+(\frac{root\_depth}{root\_dist\_param\_da})^{root\_dist\_param\_c}}$$
$$second\_term = 1 - \frac{1}{1+(\frac{root\_depth}{root\_dist\_param\_da})^{root\_dist\_param\_c}}$$
$$third\_term = 1 - \frac{root\_depth}{root\_depth\_max}$$

[SC.CRP.78]

This method relies on two crop-specific, empirically fitted parameters estimated by Fan et al., 2016: root_dist_param_c and root_dist_param_c. The default values for these parameters are those reported in the original manuscript but they are available for adjustment as user inputs.

VI. Abbreviations

Table 3: Common abbreviations used throughout this document

Abbreviation	Full Term or Phrase
ADF	Acid detergent fiber (% of DM)
ADG	Average daily weight gain (g/d)
Age1stBred	First breeding age (month)
BCS5	Body condition score (1–5 basis)
BW	Body weight (kg)
Ca	Calcium content (% of DM)
CBW	Calf birth weight (kg)
CI	Calving interval (d)
cm	centimeter
Cost	Feed cost ($/kg of DM)
CP	Crude protein (% of DM)
d	days
DE	Digestible energy (standard value, Mcal/kg)
DIM	Days in milk
DM	Dry matter (% of as-fed basis)
DMI	Dry matter intake
DOP	Days of pregnancy (d)
dRUP	RUP degradability (% of RUP)
EE	Ether extract, in crude fat (% DM)
EndCP	Endogenous CP
EndN	Endogenous N
EQSBW	Equivalent shrunk body weight (kg)
FA	Fatty acids
FIPS	Federal Information Processing Standards
g	grams
GE	Gross energy (standard value, Mcal/kg)
GrUterW	Gravid uterine weight (kg)
Kd	Rumen protein degradation rate (%/h)
kg	kilograms
kJ	kilojoule
L	liters
m	meter
max	maximum
Mcal	Megacalorie
MCP	Microbial Crude Protein
ME	Metabolizable energy (standard value, Mcal/kg)

Continued on next page

Table continued from previous page

Abbreviation	Full Term or Phrase
MICP	Microbial CP
min	minimum
mL	milliliters
MN	Microbial N
MP	Metabolizable protein
MTP	Microbial TP
MW	Mature body weight (kg)
MY	Milk yield
N	Nitrogen (% of DM)
NASEM	National Academies of Sciences, Engineering, and Medicine
NDF	Neutral detergent fiber (% of DM)
NEG	Net energy for growth (standard value, Mcal/kg)
NEL	Net energy for lactation (standard value, Mcal/kg)
NEM	Net energy for maintenance (standard value, Mcal/kg)
nNDF	Nondigestible Neutral Detergent Fiber
NPN	Non-protein Nitrogen
NRC	National Research Council (updated in 2021 to NASEM)
OM	Organic matter (% of DM)
P	Phosphorus content (% of DM)
PAF	Processing adjustment factor (not used)
Parity	Number of lactations
PrevTemp	Average daily temperature of last month (℃)
RDP	Ruminally degraded CP
ROM	Residual OM (% of DM)
RUP	Ruminally undegraded CP
s	seconds
TAN	Total ammoniacal N
TDN	Total digestible nutrient (% of DM)
TP	True protein (% of DM)
USDA	United States Department of Agriculture
UterW	Uterine weight (kg)
WSC	Water Soluble Carbohydrates

References for Crop and Soil Module

Fan, J. et al. (2016). “Root distribution by depth for temperate agricultural crops”. In: *Field Crops Research* 189, pp. 68–74. DOI: 10.1016/j.fcr.2016.02.013. URL: https://doi.org/10.1016/j.fcr.2016.02.013.

Hargreaves, George H and Zohrab A Samani (1985). “Reference crop evapotranspiration from temperature”. In: *Applied engineering in agriculture* 1.2, pp. 96–99.

Hayes, Michael HB and Roger S Swift (2020). “Vindication of humic substances as a key component of organic matter in soil and water”. In: *Advances in Agronomy* 163, pp. 1–37.

McElroy, Michael B, Steven C Wofsy, and YL Yung (1977). “The nitrogen cycle: Perturbations due to man and their impact on atmospheric N2O and O3”. In: *Philosophical Transactions of the Royal Society of London. B, Biological Sciences* 277.954, pp. 159–181.

Neitsch, S.L. et al. (2011). *Soil and Water Assessment Tool Theoretical Documentation Version 2009*. Tech. rep. Technical Report No. 406. Texas Water Resources Institute.

Parton, William J et al. (1987). “Analysis of factors controlling soil organic matter levels in Great Plains grasslands”. In: *Soil Science Society of America Journal* 51.5, pp. 1173–1179.

University of Missouri Extension (2022). *Nitrogen in the Environment: Ammonia Volatilization*. Publication WQ257, University of Missouri Extension. Reviewed November 2022; accessed 2023. URL: https://extension.missouri.edu/publications/wq257.

Vadas, P.A. et al. (2007). “A model for phosphorus transformation and runoff loss for surface-applied manures”. In: *Journal of Environmental Quality* 36.1, pp. 324–332.

Williams, JR and RW Hann (1978). *Optimal operation of large agricultural watersheds with water quality restraints*. Tech. rep. Texas Water Resources Institute.